

Hybrid AI-Assisted Page Replacement Framework Using PFN Prediction

Project Report

Submitted in partial fulfilment for the award of degree of
Bachelor of Technology
In
Department of Computer Science & Engineering



Final Year Project Report

Submitted to:

Mr. Dinesh Soni Sir
Ms. Nirmala Sharma Ma'am

Submitted by:

Hemant Singh Chouhan (22/420)
Naresh Parmar (22/519)
Naman Sehra (22/518)
Shubham Garg (23/776)

Sem / Year: VII / IV
Academic Session: 2025–2026

Department of Computer Science & Engineering
University Department, Rajasthan Technical University, Kota

CERTIFICATE

This is to certify that the project report entitled “**Hybrid AI-Assisted Page Replacement Framework Using PFN Prediction**”, submitted by:

- Hemant Singh Chouhan (22/420)
- Naresh Parmar (22/519)
- Naman Sehra (22/518)
- Shubham Garg (23/776)

in partial fulfillment of the requirements for the degree of **Bachelor of Technology in Computer Science & Engineering** at **Rajasthan Technical University, Kota**, is a record of original work carried out by them under the supervision and guidance of:

Mr. R. S. Sharma, Head of Department, CSE **Ms. Mithilesh Sharma**, Faculty, CSE Department

Mr. R. S. Sharma

Head of Department, Computer Science
(*Supervisor*)

Signature: _____

Ms. Mithilesh Sharma

Faculty, Department of Computer Science
(*Co-Supervisor*)

Signature: _____

Department of Computer Science & Engineering
Rajasthan Technical University, Kota

DECLARATION

We hereby declare that the project report titled “**Hybrid AI-Assisted Page Replacement Framework Using PFN Prediction**” is an authentic record of work carried out by us during the academic session 2025–2026.

We further declare that:

- This work has not been submitted to any other university or institution,
- All information and data sources have been duly acknowledged,
- The work embodies the results of our own study and research.

Hemant Singh Chouhan

Signature: _____

Naresh Parmar

Signature: _____

Naman Sehra

Signature: _____

Shubham Garg

Signature: _____

Department of Computer Science & Engineering
Rajasthan Technical University, Kota

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to **Mr. R. S. Sharma**, Head of Department, Computer Science, for providing continuous support, guidance, and academic resources for this project.

We extend heartfelt thanks to our **Co-Supervisor, Ms. Mithilesh Sharma**, for her constant guidance, technical insight, and encouragement, which greatly helped us during the AI model development, kernel integration, data analysis, and report preparation.

We also thank all faculty members, friends, and family for their continuous motivation and support throughout this project.

Team Members:

Hemant Singh Chouhan

Naresh Parmar

Naman Sehra

Shubham Garg

ABSTRACT

Operating systems traditionally rely on algorithms such as LRU, FIFO, or CLOCK for page replacement, but these algorithms suffer due to the absence of future knowledge about memory access patterns. This project proposes a Hybrid AI-assisted approach that predicts future PFN sequences using a multi-head GRU-based model (Version-3). We designed and deployed a Linux kernel tracking module using Kprobes to capture real-time PFN major fault sequences, processed this data for model training using nibble slicing and delta encoding, and integrated the predictor with an AI daemon and a GUI-based monitoring tool. The Version-3 model demonstrates competitive performance, enabling hybrid assistance to classical page replacement without replacing any traditional algorithms.

CONTENTS

1	Introduction	1
1.1	Background	1
1.2	OS Memory Management Overview	1
1.3	Page Faults and Performance Impact	1
1.4	Limitations of Traditional Page Replacement	2
1.5	Motivation for AI-Assisted Prediction	2
1.6	Problem Statement Overview	2
1.7	Objectives of the Project	3
1.8	Scope of Work	3
1.9	Organization of the Report	3
2	Formal Problem Statement	5
2.1	Definition of the Problem	5
2.2	System Boundaries	5
2.3	Assumptions & Constraints	6
2.4	Expected Outcomes	6
3	Requirement Analysis	7
3.1	Introduction	7
3.2	Functional Requirements	7
3.3	Non-Functional Requirements	8
3.4	Hardware and Software Requirements	9
3.5	Data Requirements	10
3.6	Constraints and Limitations	10
4	Literature Review	12
4.1	Introduction	12
4.2	Classical Page Replacement Algorithms	12
4.3	AI and Machine Learning in Systems Research	13
4.4	Predictive Page Replacement Approaches	13
4.5	Kernel Instrumentation Research	14
4.6	Identified Research Gaps	14
5	System Architecture and Design	16
5.1	Introduction	16
5.2	Baseline Architecture	16
5.3	Final Architecture	16
5.4	Data Flow Overview	18
5.5	Predictor-Assisted Page Replacement Logic	20
5.6	Trust Controller	20
5.7	Prediction Chain Builder	20
5.8	Reality Daemon Workflow	20
6	UML Diagrams and Modeling	22
6.1	Introduction	22
6.2	Use-Case Diagram	22
6.3	Activity Diagram	23
6.4	Sequence Diagram	24
6.5	Data Flow Diagram (DFD)	24
6.6	Deployment Diagram	25

7	Kernel-Level Data Acquisition	27
7.1	Introduction	27
7.2	Need for Kernel Instrumentation	27
7.3	Kprobes Overview	27
7.4	Probe Placement and Function Targets	27
7.5	PFN Major Fault Extraction Pipeline	29
7.6	Victim Selection Tracking	29
7.7	Data Fields Captured	29
7.8	CSV Logging and Persistent Storage	30
7.9	Challenges and Solutions	30
8	Data Processing and Feature Engineering	31
8.1	Introduction	31
8.2	Characteristics of PFN Traces	31
8.3	PFN Nibble Slicing	31
8.4	Sliding-Window Segmentation ($30 \rightarrow 5$)	32
8.5	Delta Encoding Strategy	32
8.6	Auxiliary Feature Engineering	32
8.7	Label Generation	33
8.8	Dataset Statistics and Preparation	33
8.9	Example Input Format	33
9	Machine Learning Model (Version-3)	34
9.1	Introduction	34
9.2	Model Objectives	34
9.3	Input Representation	34
9.4	Feature Embeddings for Auxiliary Signals	34
9.5	Model Architecture Overview	35
9.6	GRU Backbone	35
9.7	Multi-Head Output Architecture	35
9.8	Loss Functions	36
9.9	Training Configuration	36
9.10	Performance Metrics	36
9.11	Results	36
9.11.1	TOP Slice	36
9.11.2	S4 Slice	36
9.11.3	S3 Slice	36
9.11.4	S2 Slice	37
9.11.5	S1 Slice	37
9.11.6	S0 Slice	37
9.12	Performance Interpretation	37
10	Model Performance & Evaluation	38
10.1	Evaluation Metrics	38
10.2	Classification Report — TOP	38
10.3	Classification Report — S4	38
10.4	Classification Report — S3	39
10.5	Classification Report — S2	39
10.6	Classification Report — S1	39
10.7	Classification Report — S0	39
10.8	Performance Discussion	40
10.9	Strengths & Weaknesses	40

11	Integration with Linux Kernel: AI Daemon and Netlink Bridge	41
11.1	Introduction	41
11.2	Hybrid Integration Philosophy	41
11.3	System Components	41
11.4	Netlink Communication Mechanism	41
11.5	AI Daemon: Architecture and Functionality	42
11.5.1	Daemon Workflow	42
11.6	Prediction Chain Reconstruction	43
11.7	Kernel-Side Consumption of Predictions	43
11.8	Safety and Reliability Considerations	43
11.9	Module-Level Testing	43
12	Software Design and Graphical User Interface (GUI)	45
12.1	Introduction	45
12.2	Design Objectives	45
12.3	Technology Stack	45
12.4	GUI Architecture	45
12.5	Dashboard Screen	46
12.6	Memory Activity Visualization	46
12.7	Live Kernel Log Viewer	47
12.8	Prediction Visualization Panel	47
12.9	Process Detail Panel	48
12.10	User Experience Considerations	48
13	System Integration	49
13.1	Kernel + AI + GUI Integration	49
13.2	Latency Considerations	49
13.3	Real-Time Prediction Behavior	49
13.4	Integrated Workflow	50
13.5	Performance Observations	50
14	Testing & Validation	51
14.1	Unit Tests	51
14.2	Kernel Probe Stability Tests	51
14.3	AI Model Testing	52
14.4	Stress Testing	52
14.5	Integration Testing	52
14.6	Result Verification	53
15	Results & Insights	54
15.1	Major Findings	54
15.2	Reduction in False Victim Selection	54
15.3	PFN Pattern Understanding	54
15.4	Visualization of PFN Curves	55
15.5	Real-Time Logging Evidence	55
16	Comparative Analysis	56
16.1	LRU vs Hybrid AI Model	56
16.2	CLOCK/FIFO vs Predictor-Assisted Model	56
16.3	Hybrid Benefits	57
16.4	Comparison with ML Prefetch Literature	57
17	Ethical, Security & Industrial Impact	58
17.1	Kernel Safety Considerations	58
17.2	Privacy & Data Handling	58
17.3	Industrial Applications	59
17.4	Deployment Implications	59

Conclusion & Future Scope	60
Summary of Work	60
Limitations	60
Version-4 Enhancement: RESAT-X	60
Planned V4 Pipeline	61
Long-Term Research Scope	61
References	62
A Model Summary Output	63
Model Architecture Summary	63
B Kernel Module Snippets	64
Kprobe Registration	64
Victim Tracking Snippet	64
C GUI Screenshots	65
Dashboard View	65
AI Insights Panel	65
Kernel Activity Tracker	66
Prediction Overlay Graph	66
D Additional Graphs	67
PFN Curve Sample	67
Predicted vs Actual PFN Comparison	67
Event Latency Distribution	68
Workload Comparison Charts	68

LIST OF FIGURES

5.1	Baseline Architecture Conceptualization	17
5.2	Final System Architecture	18
5.3	Data Flow Architecture of the Hybrid System	19
5.4	Workflow of the Reality Daemon	21
6.1	Use-Case Diagram of the Hybrid AI-Assisted Page Replacement System	22
6.2	Activity Diagram of the System Workflow	23
6.3	Sequence Diagram Illustrating Real-Time Prediction Flow	24
6.4	Data Flow Diagram of the Hybrid System	25
6.5	Deployment Diagram Showing Integration of Kernel, AI Daemon, and GUI	26
11.1	Netlink-based integration of kernel PFN tracking with user-space AI Daemon	44
12.1	Dashboard Screen Showing System Overview	46
12.2	PFN Transition Graph from Kernel Data	46
12.3	Kernel Log Viewer Displaying PFN Major Fault Events	47
12.4	Prediction Panel Showing ML Model Output	48

INTRODUCTION

1.1 Background

Modern computing systems rely extensively on efficient memory management to deliver a seamless user experience. With the growth of multitasking operating systems, high-performance applications, and resource-intensive software such as browsers, virtual machines, game engines, and integrated development environments (IDEs), the demand for optimized memory behavior has increased significantly.

Operating systems utilize a virtual memory abstraction where each process perceives a continuous logical address space independent of physical RAM. This abstraction enhances portability, isolation, and security, but it introduces the necessity for robust memory translation and allocation strategies. The gap between virtual memory demands and the limited physical memory capacity is bridged through a paging mechanism, where memory is segmented into small, fixed-size units called *pages*.

When a requested page is not present in RAM, the CPU triggers a *page fault*. Handling a page fault is among the most expensive OS operations. It involves walking through page tables, invoking kernel routines, performing disk I/O, and potentially suspending the running process. Thus, improving the behavior of page faults has a direct positive impact on system responsiveness.

1.2 OS Memory Management Overview

The heart of virtual memory management lies in the page replacement subsystem. When RAM becomes saturated and a new page must be loaded, the OS must choose an existing page to evict. This decision is performed by algorithms such as:

- Least Recently Used (LRU)
- First-In First-Out (FIFO)
- Not Recently Used (NRU)
- CLOCK or Second-Chance
- Variants of Active/Inactive lists used in Linux

These algorithms depend on heuristics built upon past access behavior. Even Linux, despite its sophisticated Bookkeeping (active, inactive and unevictable lists), still fundamentally relies on recency/frequency signals generated from historical data.

A crucial limitation persists: **the OS still has no knowledge of future page references**. It cannot anticipate whether a page that appears unused in recent history will shortly be accessed again. As a result, every page replacement algorithm is reactive, not predictive.

1.3 Page Faults and Performance Impact

A page fault can be categorized into:

- **Minor Page Fault** – When the page is already present in memory but needs minor mapping adjustments.
- **Major Page Fault** – When the actual page content must be loaded from disk or swap.

Major page faults are extremely costly due to disk access and scheduling overhead. The performance degradation manifests as:

- Increased application loading time
- UI lags and micro-stutters
- Higher CPU wait cycles

- Reduced multitasking capability
- Battery and energy inefficiency (in portable systems)

Heavy workloads such as web browsers (with multiple processes and threads), compilers, and virtual machines frequently generate unpredictable memory usage spikes, intensifying the impact of inefficient page replacement behavior.

1.4 Limitations of Traditional Page Replacement

Although traditional algorithms like LRU and CLOCK are widely used due to their simplicity and robustness, they inherently rely on backward-looking information. Their fundamental limitations include:

- **No prediction of future PFN chains:** They depend only on past recency or frequency.
- **Susceptibility to irregular workloads:** Sudden process spikes disrupt heuristics.
- **One-policy-fits-all:** They cannot dynamically adapt to application-specific patterns.
- **High false victim selection:** Pages evicted earlier may be needed shortly afterward.

This creates a performance bottleneck where the OS repeatedly makes suboptimal eviction decisions, especially under frequent, unpredictable major page faults.

1.5 Motivation for AI-Assisted Prediction

Artificial Intelligence, particularly sequence modeling methods (LSTM/GRU), has the capability to analyze temporal patterns and forecast future values based on historical context. Applying ML to kernel memory behavior opens up a unique opportunity:

If the OS could anticipate upcoming page accesses, it could avoid evicting pages that will be needed soon and preemptively prepare memory for incoming page faults.

Instead of replacing classical algorithms, the concept is to develop a **Hybrid AI-Assisted Page Replacement System** that:

- Models PFN major fault sequences,
- Predicts a chain of future page references,
- Shares these predictions with the OS,
- Helps the page replacement subsystem validate and refine decisions.

This provides the OS with the missing component: **future awareness**.

1.6 Problem Statement Overview

Despite technological advancements, the page replacement subsystem in the Linux kernel still suffers because it cannot see beyond current access patterns. The precise problem addressed in this project is:

“To design a predictive mechanism that forecasts future PFN major fault patterns so that the OS can reduce false victim selection, minimize latency, and improve overall memory management efficiency.”

The problem is technically challenging because:

- PFN sequences are irregular and influenced by kernel internals,
- Data extraction from the kernel requires low-level instrumentation,
- Prediction requires multi-step forecasting with structured outputs,
- Integration must not compromise OS stability or performance.

1.7 Objectives of the Project

This project focuses on bridging the gap between operating system design and modern machine learning capabilities. Its primary objectives are:

- To extract real-time PFN major fault sequences from the Linux kernel using Kprobes.
- To design a feature engineering pipeline using nibble-based PFN decomposition and delta encoding.
- To build a multi-head GRU-based predictive model (Version-3) capable of forecasting multiple future slices.
- To integrate the model in real time using an AI daemon communicating with the kernel via Netlink.
- To assist—rather than replace—existing page replacement logic.
- To develop a full desktop GUI for performance monitoring and visualization.

1.8 Scope of Work

The scope of the project includes:

- Low-level kernel instrumentation and data collection,
- Real-time PFN trace processing,
- Sequence modeling using deep learning,
- Real-time prediction through an AI daemon,
- Development of a visual monitoring interface,
- System integration and benchmarking.

The project intentionally does **not** modify the internal LRU logic of Linux. Instead, it operates as an advisory component that enhances decision quality.

1.9 Organization of the Report

The remainder of this report is structured as follows:

- **Chapter 2:** Formal definition of the problem and boundaries.
- **Chapter 3:** Requirement analysis including system, functional, and data requirements.
- **Chapter 4:** Review of related research and OS/ML literature.
- **Chapter 5:** System architecture including monitoring, prediction, and decision flows.
- **Chapter 6:** UML modeling and workflows.
- **Chapter 7:** Kernel-level PFN data tracking technique using Kprobes.
- **Chapter 8:** Feature engineering, nibble slicing, delta encoding, and dataset preparation.
- **Chapter 9:** Detailed explanation of the Version-3 GRU model.
- **Chapter 10:** Performance evaluation across all PFN slices.
- **Chapter 11:** AI daemon and Netlink integration design.
- **Chapter 12:** GUI implementation overview.
- **Chapter 13:** System-level integration results.
- **Chapter 14:** Testing and validation strategies.

- **Chapter 15:** Observed insights and graphical results.
- **Chapter 16:** Comparative analysis with traditional approaches.
- **Chapter 17:** Ethical, security, and industrial implications.
- **Chapter 18:** Conclusion, limitations, and future improvements including Version-4 (RESAT-X).

FORMAL PROBLEM STATEMENT

2.1 Definition of the Problem

The performance of a modern multitasking operating system is significantly influenced by the behavior of its page replacement subsystem. When the physical memory is full and a new page must be loaded, the OS selects a victim page to evict. Traditional page replacement algorithms (e.g., LRU, FIFO, CLOCK) base this decision entirely on heuristics derived from historical reference patterns. However, these algorithms consistently suffer from a fundamental limitation:

They operate without any knowledge of the application’s future page access behavior.

As a consequence, the OS may evict pages that will soon be required again, leading to:

- Increased major page faults,
- Higher latency due to disk or swap I/O,
- CPU and kernel pipeline stalls,
- Reduced application responsiveness,
- Poor multitasking performance under heavy workloads.

The central problem addressed in this project is therefore defined as follows:

“To design a predictive system capable of forecasting the future Page Frame Number (PFN) major fault sequence, and assisting the OS page replacement algorithm in reducing false victim selections and improving overall memory performance.”

This predictive system must operate in real-time, remain lightweight, and integrate safely with the kernel without modifying core OS mechanisms.

2.2 System Boundaries

To ensure clarity in design and implementation, the boundaries of the system are explicitly defined:

Within Scope

- Extraction of PFN major fault traces using Linux Kprobes.
- Monitoring memory access behavior at the kernel level.
- Construction of an ML-ready dataset using nibble slicing and delta encoding.
- Training a multi-head GRU-based predictor (Version-3).
- Deploying a user-space AI daemon capable of real-time inference.
- Communicating predictions to the kernel via Netlink.
- Performing victim validation using predicted future PFN chains.
- Developing a GUI for visualization and monitoring.

Out of Scope

- Modifying or replacing the native Linux LRU page replacement policy.
- Direct manipulation of page tables or core memory management.
- Predicting minor faults or system-level IRQ-driven memory events.
- Performing unsafe operations inside kernel space.
- Integrating unverified machine learning decisions directly into victim selection logic.

These boundaries ensure that the project remains safe, practical, and academically sound.

2.3 Assumptions & Constraints

The design and execution of the project rely on a set of assumptions and constraints:

Assumptions

- The PFN major fault sequence contains enough structure to be learnable with ML.
- The kernel Kprobe instrumentation does not introduce noticeable overhead.
- The AI model (Version-3) can produce stable 5-step predictions for each nibble slice.
- Netlink allows reliable low-latency communication between kernel space and user space.
- The GUI system runs asynchronously and does not interfere with kernel or model performance.

Constraints

- The kernel cannot be modified beyond the addition of tracking modules.
- Prediction latency must be low enough to operate in real-time.
- The system must handle high-frequency fault events without bottlenecks.
- All ML pipelines must run under limited memory and CPU budgets.
- Training datasets are constrained by the nature of real kernel workloads.

These assumptions and constraints guide architectural decisions and ensure stability of the prototype in real-world use.

2.4 Expected Outcomes

The expected outcomes of the project are summarized below:

- A functioning kernel-level module capable of extracting accurate PFN major fault traces.
- A complete dataset representing real-world PFN slices, delta transitions, and auxiliary features.
- A high-performance GRU-based model (Version-3) capable of multi-step prediction of PFN slices.
- An AI daemon capable of receiving kernel events, performing inference, and providing predictions.
- A real-time prediction pipeline integrated using Netlink sockets.
- Reduction in false victim selections based on model-validated future context.
- A user-friendly GUI to monitor predictions, system behavior, and data flow.
- A foundation for future enhancement via Version-4 incorporating RESAT-X smoothing (tested but not yet integrated).

Collectively, these outcomes demonstrate the feasibility of integrating machine learning into operating system memory management workflows, without modifying the core eviction logic.

REQUIREMENT ANALYSIS

3.1 Introduction

Requirement analysis serves as the foundation for system design, architecture, and implementation. For a hybrid AI-assisted page replacement system, it is essential to clearly identify what the system must do (functional requirements), the quality and performance expectations (non-functional requirements), and the constraints imposed by kernel-level instrumentation and real-time prediction workflows.

This chapter provides a detailed breakdown of all requirements necessary for successful development, integration, and evaluation of the proposed system.

3.2 Functional Requirements

Functional requirements describe explicit behaviors and operations that the system must support.

FR1: Kernel-Level PFN Tracking

The system must capture real-time Page Frame Number (PFN) major fault sequences by instrumenting the Linux kernel using Kprobes. It must record:

- PID and process name (COMM),
- Virtual Address (VA),
- PFN and its nibble slices,
- Mapping and folio index,
- Fault timestamp and latency.

FR2: Data Logging and Export

The tracked fault records must be exported to a structured format (CSV) for:

- Offline dataset preparation,
- Model training,
- Real-time monitoring,
- Debugging and visualization.

FR3: Feature Engineering Pipeline

The system must transform raw PFN sequences into a machine-learning-ready dataset by:

- Performing nibble slicing (TOP, S4, S3, S2, S1, S0),
- Applying sliding-window segmentation (30 input steps $\rightarrow$ 5 output steps),
- Encoding auxiliary features (PID, mapping, folio, latency),
- Building target labels for each nibble slice.

FR4: Multi-Head GRU Model Training (Version-3)

The system must train a machine learning model capable of:

- 5-step sequential forecasting,
- Predicting all 6 PFN nibble slices simultaneously,
- Producing classification logits for each nibble category,
- Supporting KL-consistency between pre-softmax projections.

FR5: AI Daemon Operation

A user-space daemon must:

- Receive kernel-triggered major fault events,
- Preprocess the input sequence,
- Run inference using Version-3 model,
- Generate future PFN prediction chains,
- Send prediction data back to kernel space through Netlink.

FR6: Victim Validation

The system must assist the OS by validating the kernel-selected victim using predicted future PFN chains. If the victim appears in the predicted sequence, the AI can suggest revisiting the selection logic.

FR7: GUI Visualization

The GUI must:

- Display real-time PFN sequences,
- Show AI predictions,
- Visualize memory load and kernel activity,
- Provide user-friendly monitoring views.

3.3 Non-Functional Requirements

Non-functional requirements define the performance, reliability, and quality expectations of the system.

NFR1: Real-Time Performance

Prediction latency must be low enough that the AI daemon responds within the timeframe acceptable to kernel-level decision cycles.

NFR2: System Stability

Kernel modules must not interfere with:

- Kernel memory safety,
- Scheduling,
- System responsiveness.

NFR3: Minimal Overhead

Kprobe instrumentation must introduce negligible performance overhead and must be enabled/disabled safely.

NFR4: Scalability

The model must handle:

- High-frequency page fault bursts,
- Multiple processes triggering events,
- Multi-core system behavior.

NFR5: Reliability

The system must:

- Handle invalid or incomplete kernel messages,
- Recover gracefully from daemon restarts,
- Ensure no data corruption occurs.

NFR6: Security and Safety

Since kernel instrumentation is sensitive:

- All access must require root privileges,
- No unsafe memory operations are allowed,
- AI predictions must not directly modify kernel memory.

NFR7: Maintainability

The project must be:

- Well-structured,
- Modular,
- Easy to debug and extend,
- Compatible with future enhancements (e.g., RESAT-X in Version-4).

3.4 Hardware and Software Requirements

Hardware

- Multi-core processor (min 4 cores recommended)
- Minimum 8 GB RAM (16 GB preferred)
- SSD storage for faster swap/page operations

Software

- Linux Kernel 5.x or above
- GCC, Make, Kernel Headers (for module compilation)
- Python 3.8+ for AI daemon and model training
- TensorFlow/Keras for GRU-based model development
- Netlink libraries for kernel-user communication
- PySide2/PyQt5 for GUI

3.5 Data Requirements

DR1: Raw Kernel Logs

The PFN fault stream must include:

- VA, PFN, PID, COMM
- Mapping and folio index
- Fault latency (nanoseconds)

DR2: ML Dataset

Prepared using:

- Windowed sequences (30-step input)
- Target prediction horizon (5 steps)
- One-hot encoded labels for each nibble slice
- Auxiliary numerical feature embeddings

DR3: Evaluation Data

A separate validation dataset must be generated from real workloads with no data leakage.

3.6 Constraints and Limitations

C1: Kernel Access Restrictions

Direct modification of memory-handling logic is not allowed. Only safe monitoring points may be instrumented.

C2: Unpredictable Workloads

Real system behavior may include:

- Sudden PFN spikes,
- Irregular patterns,
- Highly process-specific structures,

making prediction inherently challenging.

C3: ML Model Accuracy Limits

Lower nibble slices (S0, S1) are more random and show lower accuracy.

C4: Resource Constraints

The AI daemon must operate under:

- Limited CPU usage,
- Strict latency windows,
- Real-time constraints.

C5: Dataset Variability

PFN behavior varies by:

- Hardware,
- Kernel version,
- Running applications,
- System load.

C6: Ethical and Safety Restrictions

No part of the system may:

- Compromise user privacy,
- Log sensitive user memory contents,
- Perform unsafe kernel overrides.

LITERATURE REVIEW

4.1 Introduction

A literature review is essential for understanding existing work related to memory management, page replacement algorithms, system performance optimization, and the integration of machine learning techniques in operating system behavior prediction. This chapter presents a detailed analysis of prior research, enabling the identification of gaps and motivating the need for a hybrid AI-assisted page replacement framework.

4.2 Classical Page Replacement Algorithms

Page replacement algorithms have been extensively studied since early computer architectures. The most well-known classical techniques include:

1. FIFO (First-In First-Out)

One of the simplest strategies, FIFO evicts the page that has been in memory the longest. However, it often performs poorly due to:

- Lack of recency awareness,
- Belady's anomaly — where increasing memory worsens performance,
- Inability to adapt to workload patterns.

2. LRU (Least Recently Used)

LRU replaces the page that has not been accessed for the longest period of time. It is widely regarded as more efficient than FIFO, but suffers from:

- High overhead due to tracking recency,
- Inaccurate prediction during bursty workloads,
- Poor handling of sequential-scanning workloads.

3. CLOCK / Second-Chance Algorithm

CLOCK approximates LRU by using a circular list and a reference bit. Although more scalable, it still:

- Depends purely on historical information,
- Fails to handle unpredictable future access spikes,
- Suffers under workloads with multiple rapidly changing working sets.

4. Linux LRU Variants

Linux uses an enhanced LRU-based design with:

- Active and inactive lists,
- Page aging,
- Reference bit scanning,
- Background reclaim processes.

Despite these improvements, the absence of future knowledge remains a major limitation. The Linux kernel still relies exclusively on **backward-looking heuristics**.

4.3 AI and Machine Learning in Systems Research

Recent research has explored the use of machine learning to augment system components.

1. Predictive Prefetching

Machine learning-based prefetchers attempt to anticipate upcoming memory accesses. While these works influence CPU cache behavior, they differ significantly from OS-level PFN prediction because:

- Cache misses operate on microsecond timescales;
- OS page faults operate on millisecond-to-microsecond scales;
- Prefetchers work on virtual memory access traces, not PFN faults;
- OS kernels do not expose fine-grained access logs by default.

2. Reinforcement Learning for Page Replacement

Some studies propose RL-based page replacement, but practical deployment is challenging because:

- RL requires millions of iterations for convergence,
- Kernel environments are nondeterministic,
- Unsafe exploration could destabilize memory subsystems,
- Model behavior is difficult to validate mathematically.

Thus, RL-based solutions remain mostly theoretical.

3. LSTM/GRU for Sequence Learning

Recurrent neural networks (RNNs), particularly GRUs and LSTMs, have proven effective in tasks such as:

- Time-series forecasting,
- System log prediction,
- Anomaly detection,
- Load forecasting,
- Network traffic prediction.

These architectures can model temporal dependencies and nonlinear patterns, making them strong candidates for predicting future PFN slice sequences.

4.4 Predictive Page Replacement Approaches

Several academic attempts have been made to use prediction in memory management:

History-Based Predictors

Some works analyze access history to predict upcoming pages. However:

- They only work well for regular patterns,
- They fail under real-world irregular workloads,
- They do not handle PFN-level behavior.

Statistical Models

Markov models and probabilistic methods have been attempted, but:

- State-space grows extremely large,
- PFN values span large ranges with non-Markovian behavior,
- Predictions break down in multi-process environments.

Deep Learning Approaches

Recent attempts involve neural networks predicting memory accesses, yet:

- Most focus on CPU cache traces, not OS PFN behavior,
- No prior work captures true PFN major faults from the Linux kernel,
- Real-time integration with the kernel is rarely explored.

Thus, the domain remains relatively underdeveloped.

4.5 Kernel Instrumentation Research

Tracking internal kernel memory events is a difficult problem due to safety restrictions. Key research includes:

DTrace, eBPF, and SystemTap

These tools allow tracing of kernel functions, but:

- They are high-level,
- They incur non-negligible overhead under continuous tracing,
- They do not provide direct PFN-level access at the exact fault site.

Kprobes

Kprobes provide:

- Function-level instrumentation,
- Low overhead,
- Precise execution interception,
- Real-time event collection.

However, PFN extraction requires:

- Identifying correct low-level kernel routines,
- Understanding internal structures such as `vmf`, `folio`, `swap_entry`,
- Manually mapping virtual to PFN transitions.

There is almost no research publicly documenting the extraction of PFN major faults using Kprobes.

4.6 Identified Research Gaps

Based on the literature reviewed, the following gaps are observed:

Gap 1: No Future Awareness in Existing Page Replacement

Traditional algorithms cannot predict future accesses.

Gap 2: Lack of Real PFN-Level Machine Learning Research

Most ML-based memory studies operate on:

- Simulated traces,
- Virtual addresses,
- Cache-level data.

PFN major fault streams from the Linux kernel are not widely studied.

Gap 3: Missing Practical Kernel Integration

Academic ML papers rarely integrate their models into live kernels, while:

- Real-time constraints,
- Prediction latency,
- Kernel–user space synchronization,

are largely unaddressed.

Gap 4: Lack of Multi-Slice PFN Modeling

PFN values are large integers (up to 32 bits), and no work decomposes PFN into nibble slices for structured multi-head prediction.

Gap 5: No GUI Tools for PFN Prediction Visualization

System tools display:

- Memory consumption,
- Swap usage,
- Process activity,

but no existing tool displays predictive PFN chains.

SYSTEM ARCHITECTURE AND DESIGN

5.1 Introduction

The proposed system follows a hybrid architectural approach combining Linux kernel instrumentation, a machine learning prediction engine, a user-space daemon, and a graphical monitoring interface. Unlike classical page replacement systems that operate exclusively on historical memory access patterns, the hybrid model incorporates predictive intelligence by providing future PFN major fault context.

This chapter presents detailed descriptions of the baseline and final architectures, data flow mechanisms, and the functional modules responsible for real-time prediction and validation.

5.2 Baseline Architecture

The initial conceptual architecture was designed before the team acquired deeper understanding of kernel mechanics and PFN-level behavior. The baseline architecture included:

- Direct extraction of the page demand sequence (virtual page access stream),
- A neural network trained on direct PFN sequences,
- A simple user-space predictor that supplied future PFN values to the kernel,
- A modular GUI for system monitoring.

This design captured the core idea of AI-assisted page replacement, but during implementation several kernel-level constraints surfaced:

- Linux does not expose direct page demand sequences to userspace,
- Capturing pre-fault virtual memory accesses is not feasible without intrusive modifications,
- PFN transitions occur at deep kernel layers inside memory fault handlers,
- Direct tracking via user-space hooks is not possible.

Thus, the baseline design was theoretically strong but practically limited.

5.3 Final Architecture

After studying Linux kernel internals and evaluating feasible instrumentation points, the architecture evolved into a practical, kernel-compatible system. The final architecture includes the following core modules:

1. **Kernel PFN Tracker using Kprobes** Instruments low-level functions like `do_swap_page`, `finish_fault`, and `shrink_folio_list` to extract PFN major fault events and victim selection attempts.
2. **Data Formatting Layer** Converts kernel event structures into a structured log format (CSV-like rows).
3. **ML Dataset Preprocessing and Feature Engineering** Uses nibble slicing, delta encoding, sliding windows, and auxiliary features for model training.
4. **Version-3 GRU-Based Prediction Model** A multi-head architecture producing 5-step predictions for all PFN nibble slices.
5. **AI Daemon (Reality Daemon)** A continuously running user-space service that:
 - Receives kernel events,
 - Performs inference,

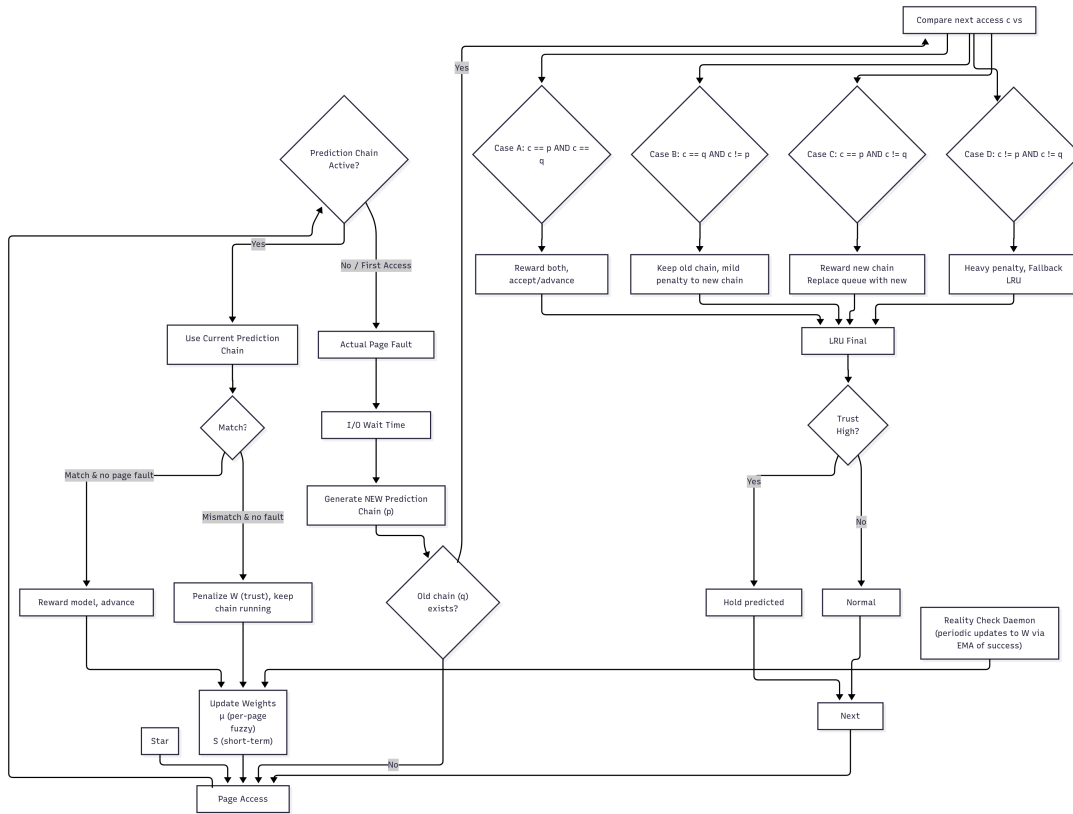


Figure 5.1: Baseline Architecture Conceptualization

- Builds prediction chains,
 - Sends guidance back to the kernel.
6. **Netlink Communication Interface** Provides bidirectional data flow between kernel and user space.
 7. **Trust Controller** Evaluates model predictions and decides whether they should influence victim validation.
 8. **Graphical User Interface** Offers real-time visual monitoring of:
 - System memory load,
 - PFN fault streams,
 - AI predictions,
 - Kernel activity.

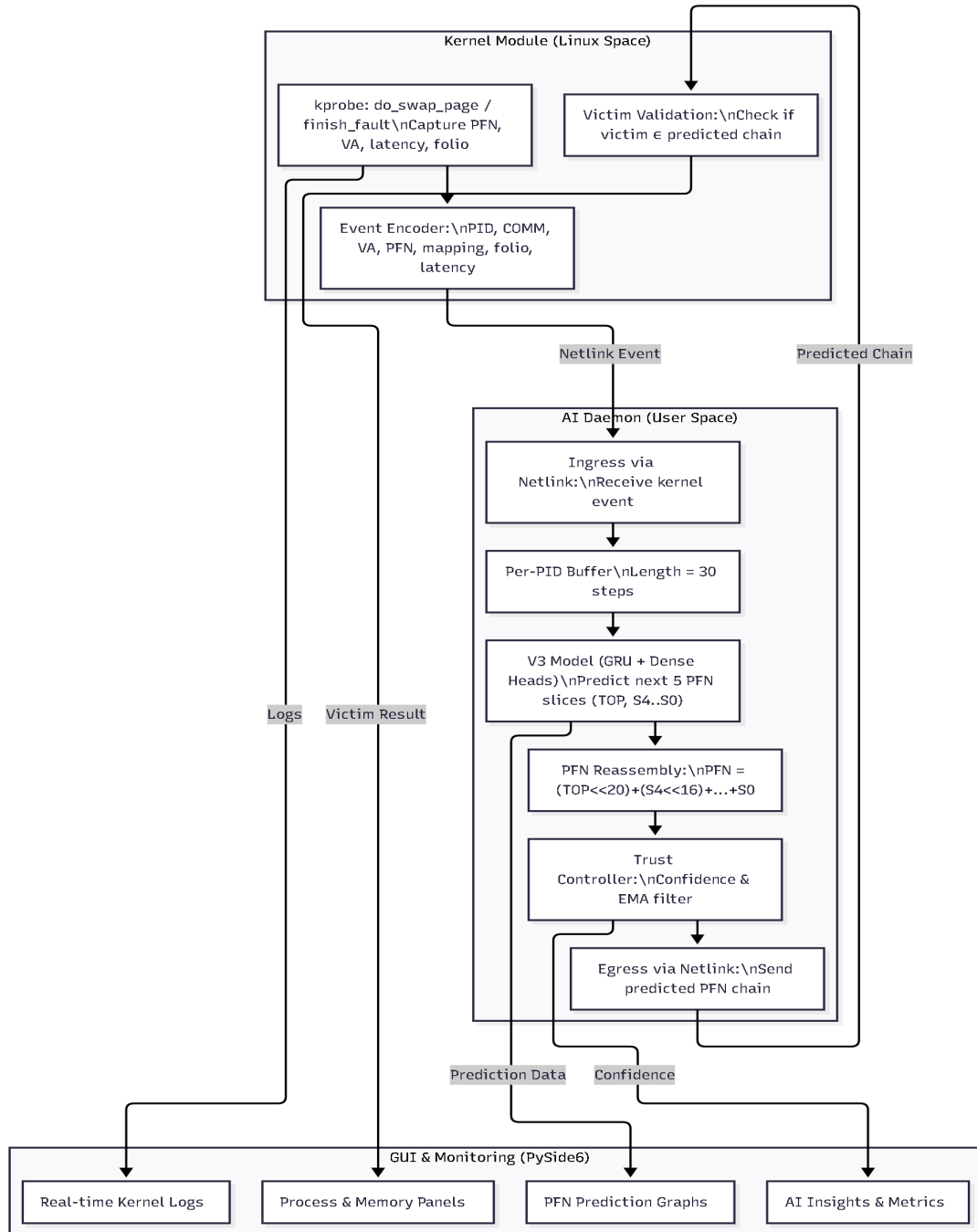


Figure 5.2: Final System Architecture

5.4 Data Flow Overview

The complete data flow of the system involves four distinct layers: kernel, data processing, prediction, and control.

1. Kernel Layer

The kernel tracking module captures the exact PFN and associated metadata during a major page fault and when a page is selected as a potential victim.

2. User-Space Data Channel

Events are transmitted to the AI daemon over the Netlink socket channel with minimal latency.

3. Prediction Layer

The AI daemon:

- Maintains per-process PFN fault history,
- Prepares sliding windows of 30 steps,
- Sends tensors into the Version-3 GRU model,
- Receives 5-step predictions for each nibble slice.

4. Decision Layer

The kernel uses predicted PFN chains to:

- Validate victim selection,
- Avoid evicting pages predicted to be accessed soon,
- Improve long-term memory performance.

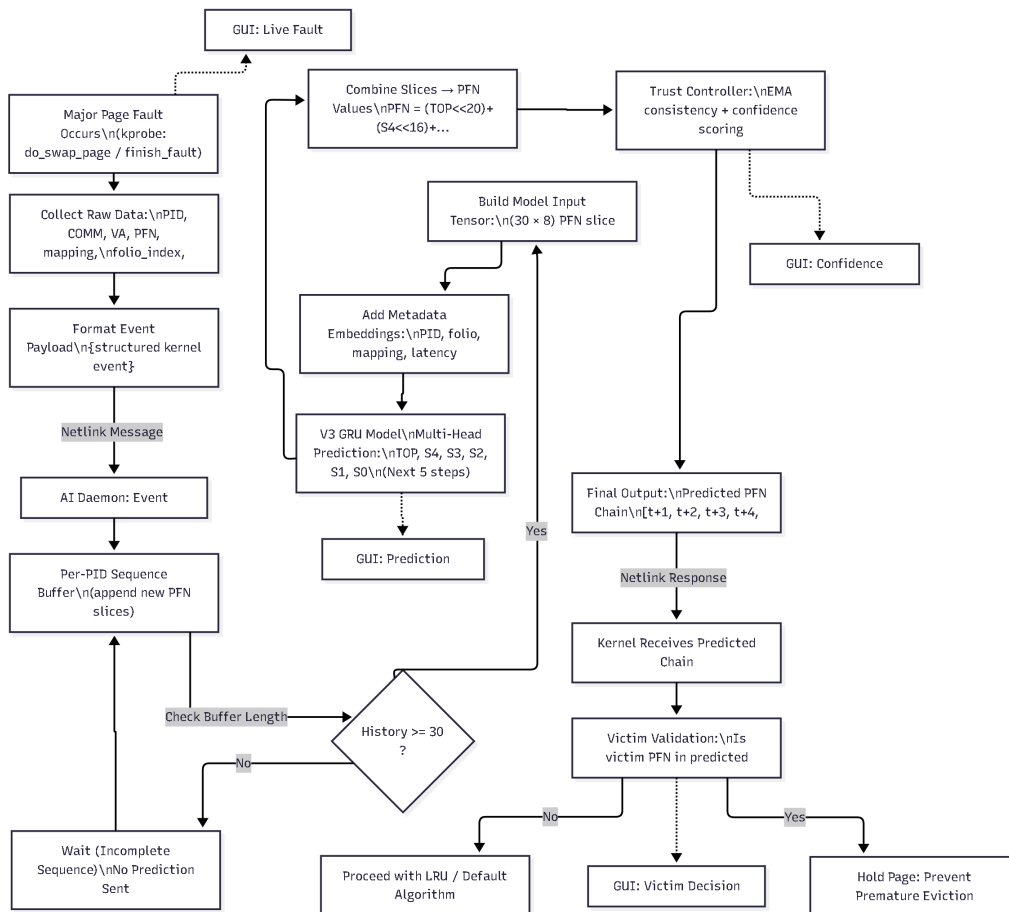


Figure 5.3: Data Flow Architecture of the Hybrid System

5.5 Predictor-Assisted Page Replacement Logic

The hybrid system does not replace the Linux page replacement algorithm. Instead, it augments the decision process by notifying the kernel when the selected victim is predicted to reappear in the near future.

The logic consists of:

1. Capture major fault event,
2. Predict next 5-step PFN chain,
3. Compare predicted chain with the current victim,
4. If match occurs:
 - Mark victim as “unsafe to evict”,
 - Allow OS to reconsider alternate candidates.
5. Otherwise, continue with traditional LRU eviction.

This method ensures safety while still offering predictive enhancement.

5.6 Trust Controller

Because machine learning predictions are probabilistic, the Trust Controller ensures stability by:

- Tracking model confidence across multiple steps,
- Using a threshold-based trust score,
- Enforcing cooldown periods after consecutive mispredictions,
- Selecting the most reliable slices (TOP/S4) for victim comparison.

The Trust Controller prevents the OS from reacting to noisy predictions and ensures that only statistically reliable guidance influences memory decisions.

5.7 Prediction Chain Builder

The multi-head model outputs 5-step predictions for each nibble slice. The chain builder:

- Reconstructs combined PFN predictions from nibble slices,
- Generates complete PFN integers,
- Produces a 5-step future PFN chain,
- Shares chain summaries with the kernel.

The chain builder is essential for converting slice-level predictions into meaningful PFN guidance.

5.8 Reality Daemon Workflow

The Reality Daemon is the central AI engine. Its workflow includes:

1. **Event Listener:** Waits for Netlink messages from the kernel.
2. **Buffer Update:** Adds new PFN events to the process-specific sequence history.
3. **Preprocessing:** Performs nibble slicing, embedding, and window formation.
4. **Inference:** Runs the GRU V3 model on the latest sequence.
5. **Chain Builder:** Constructs predicted PFN sequences.

6. **Trust Evaluation:** Decides whether predictions should influence kernel behavior.

7. **Response:** Sends validated predictions back to the kernel.

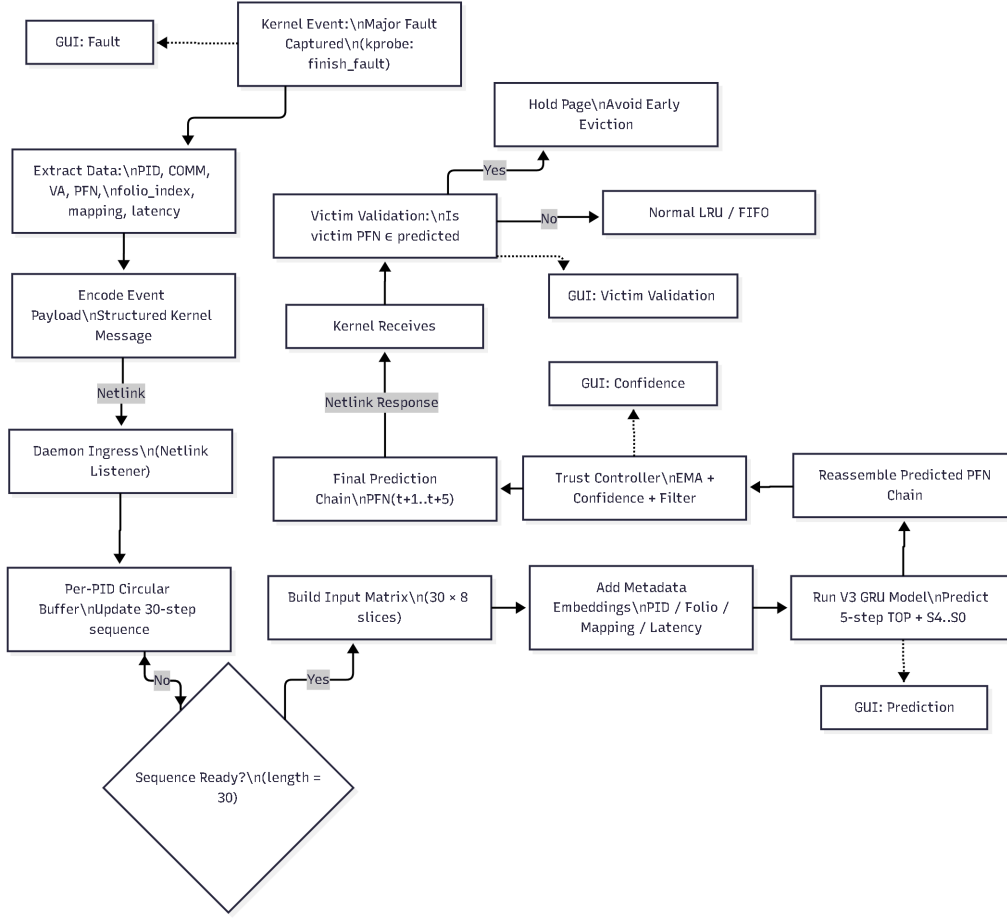


Figure 5.4: Workflow of the Reality Daemon

UML DIAGRAMS AND MODELING

6.1 Introduction

Unified Modeling Language (UML) diagrams provide a structural and behavioral view of the system. For a hybrid AI-assisted page replacement framework, UML modeling helps illustrate:

- User and system interactions,
- Internal control flows,
- Component responsibilities,
- Communication patterns,
- Deployment architecture.

This chapter presents all relevant UML and system modeling diagrams used for the design and development of the proposed system.

6.2 Use-Case Diagram

The Use-Case Diagram captures the high-level interactions among external actors (Kernel, User, AI Daemon) and the system functionalities such as PFN tracking, prediction, decision-making, and visualization.

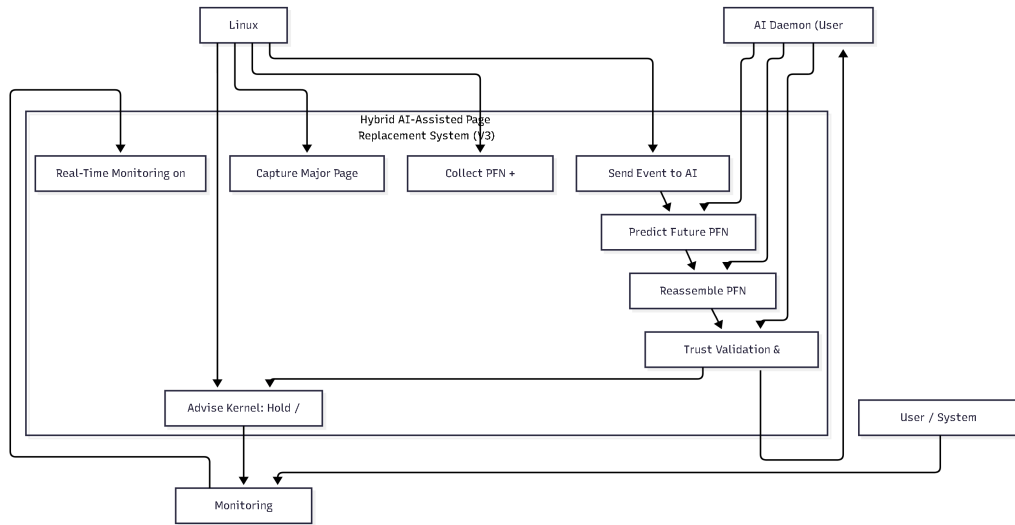


Figure 6.1: Use-Case Diagram of the Hybrid AI-Assisted Page Replacement System

Key use cases include:

- Capture PFN major fault events,
- Send PFN data to user space,
- Predict future PFN sequences,
- Validate OS victim selection,
- Display memory and prediction graphs in GUI.

6.3 Activity Diagram

The Activity Diagram represents the end-to-end operational flow of the system. It outlines how PFN events are collected from the kernel, processed, predicted, and sent back to the OS.

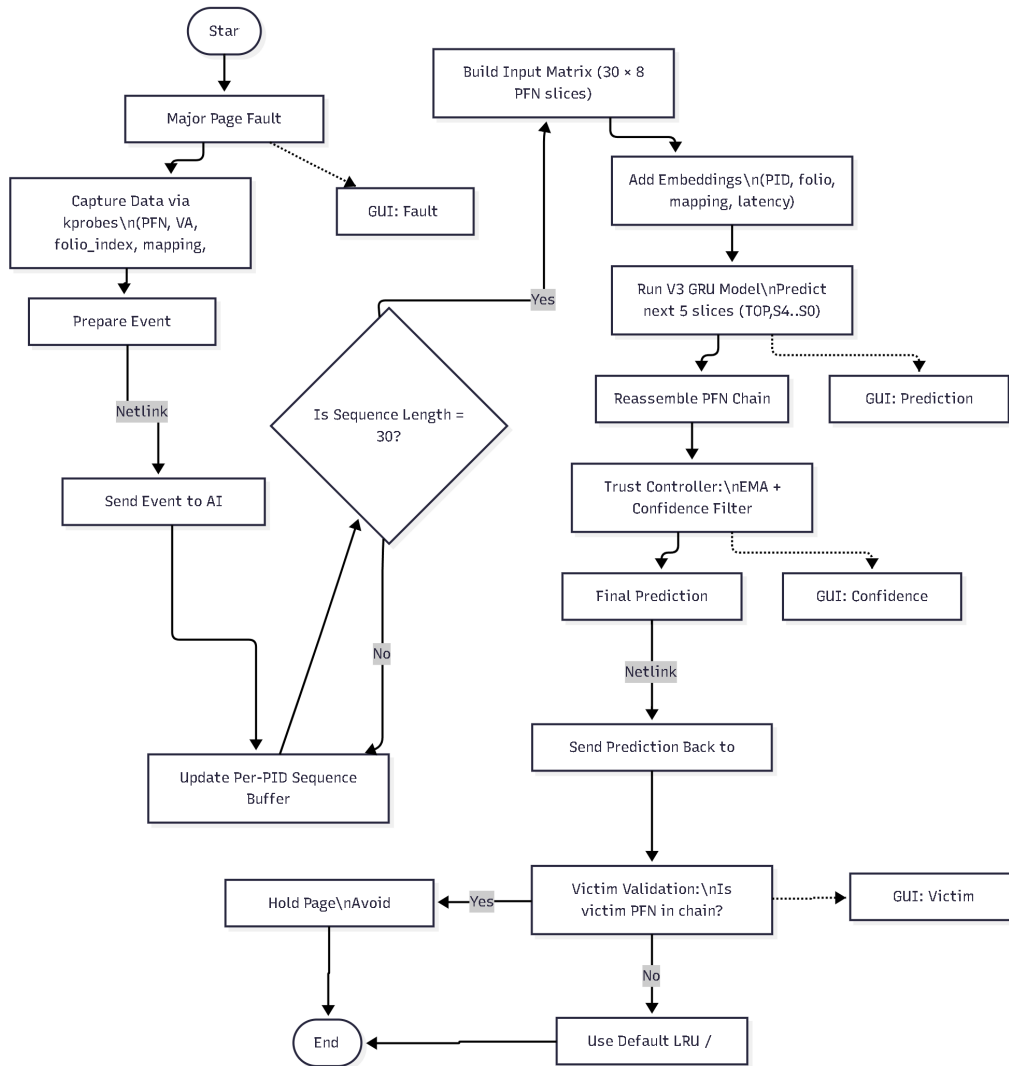


Figure 6.2: Activity Diagram of the System Workflow

The workflow includes:

1. Kernel detects major fault,
2. Kprobe logs PFN event,
3. AI Daemon receives Netlink message,
4. Preprocessing and window preparation,
5. GRU model inference,
6. Trust score evaluation,
7. Kernel receives prediction for victim validation.

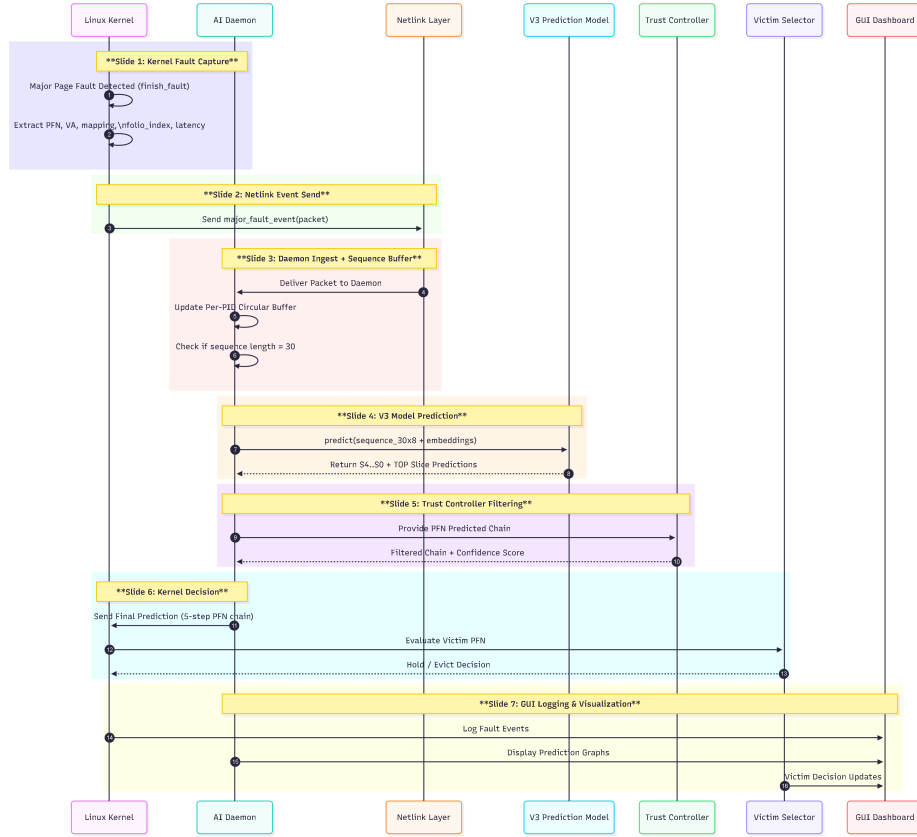


Figure 6.3: Sequence Diagram Illustrating Real-Time Prediction Flow

6.4 Sequence Diagram

The Sequence Diagram models the communication between kernel, daemon, and prediction modules at runtime. It captures chronological message flow essential for real-time prediction.

Core interactions include:

- **Kernel** → sends PFN event to Daemon,
- **Daemon** → prepares inputs and runs ML prediction,
- **Daemon** → sends predicted PFN chain to Kernel,
- **Kernel** → uses prediction to validate victim selection.

6.5 Data Flow Diagram (DFD)

The Data Flow Diagram presents the logical movement of information between system components.

Key data flows:

- PFN logs from Kernel to Daemon,
- Processed sequences and model predictions,
- Prediction chains back to kernel,
- Monitoring data forwarded to GUI.

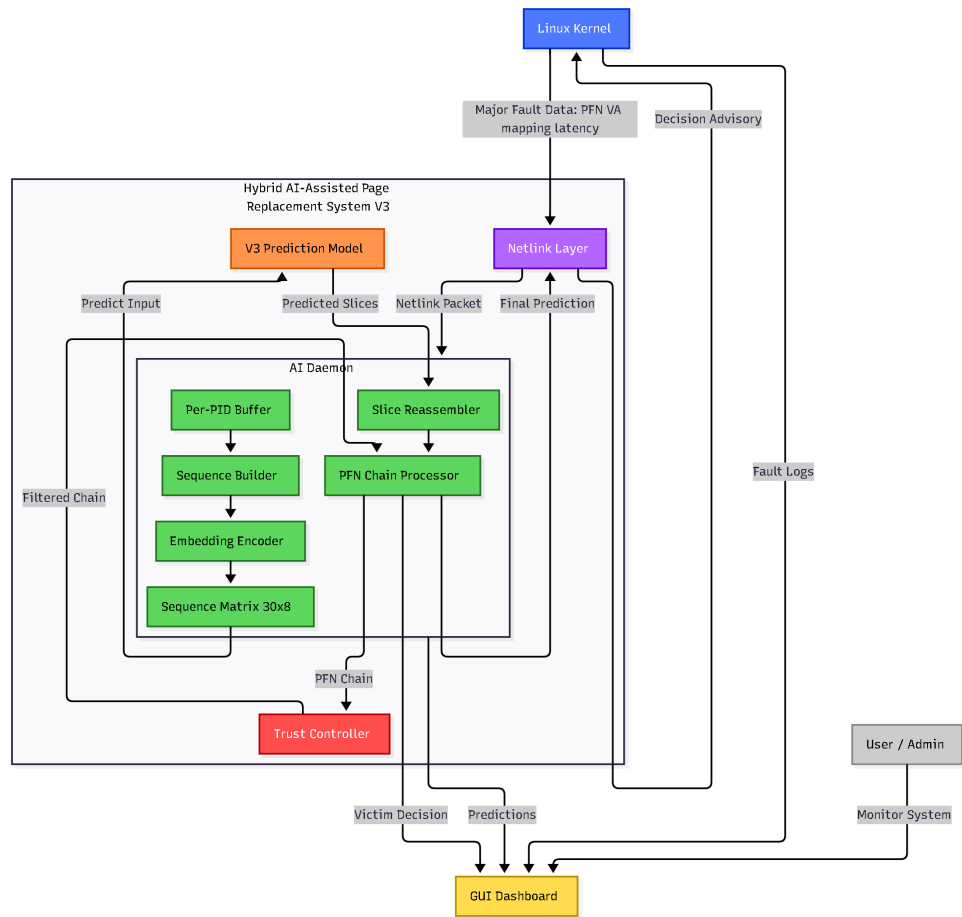


Figure 6.4: Data Flow Diagram of the Hybrid System

6.6 Deployment Diagram

The Deployment Diagram provides a structural view of how system modules are deployed on host hardware and OS components.

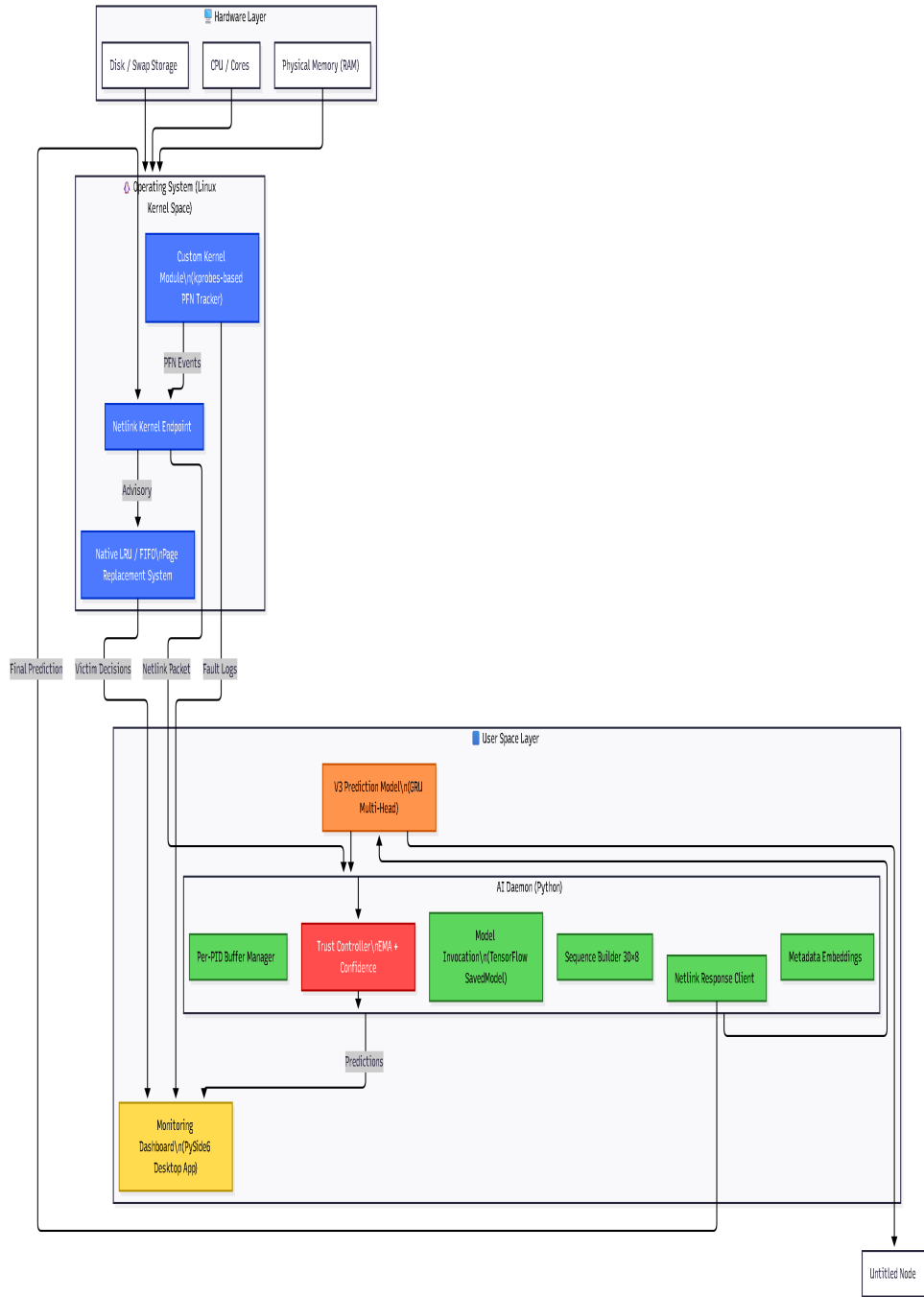


Figure 6.5: Deployment Diagram Showing Integration of Kernel, AI Daemon, and GUI

The deployed entities include:

- Kernel Probe Module (Kprobe Handler),
- User-Space AI Daemon,
- TensorFlow Runtime for GRU Model,
- GUI Application,
- Netlink subsystem for communication.

KERNEL-LEVEL DATA ACQUISITION

7.1 Introduction

Accurate prediction of future Page Frame Number (PFN) behavior requires real-time access to low-level memory events occurring inside the Linux kernel. Since the kernel does not expose page fault or page replacement internals to user space, custom instrumentation is necessary to capture the PFN major fault sequence, page removal indicators, and associated metadata such as mapping, folio index, and access latency.

This chapter explains the techniques used to extract these events using **Kprobes**, a kernel debugging and instrumentation framework that allows dynamic insertion of breakpoints into kernel functions without recompiling or restarting the system.

7.2 Need for Kernel Instrumentation

Traditional system monitoring tools (e.g., `top`, `vmstat`, `perf`) provide high-level counters but do not expose:

- The exact PFN accessed,
- The transition from VA to PFN,
- The location within the fault-handling sequence,
- The internal folio index,
- Victim selection during memory reclaim.

To train a predictive model capable of understanding PFN behavior, we require precise dataset entries collected at the exact moment of major page faults.

Only low-level instrumentation such as **Kprobes** can provide this level of granularity.

7.3 Kprobes Overview

Kprobes allow dynamically inserting handlers into kernel functions at runtime. The mechanism supports:

- **Pre-handlers** (executed before the probed instruction),
- **Post-handlers** (executed after the probed instruction),
- **Exception handlers** for resolving unexpected behavior.

Key advantages include:

- No kernel recompilation required,
- Minimal performance overhead,
- Safe removal at runtime,
- Ability to inspect kernel structures such as `vmf` and `folio`.

In this project, multiple probes were strategically placed to capture both:

- Major page fault events,
- Page eviction attempts (victim selections).

7.4 Probe Placement and Function Targets

Three primary kernel functions were instrumented:

1. `do_swap_page` / `__do_swap_page`

This function is invoked when a faulting virtual address maps to a page located in swap storage. A **pre-handler** was inserted to extract:

- PID and process name,
- Virtual Address (VA),
- Swap entry key,
- Initial timestamp for latency measurement.

2. `finish_fault`

A **post-handler** was inserted here because the PFN becomes available only after the fault completes. This captured:

- PFN (Page Frame Number),
- Mapping pointer,
- Folio index,
- Latency (difference between timestamps),
- Final processed VA.

3. `shrink_folio_list`

This function is used internally by Linux during memory reclaim. By probing the region after label `free_it:`, we captured:

- Evicted folio,
- Page being considered as victim,
- Reclaim context.

The probes registered in the module were:

```
/* Swap Fault Tracing */
static struct kprobe kp_swap_start = {
    .symbol_name = "do_swap_page",
    .pre_handler = do_swap_pre,
};

static struct kprobe kp_swap_done = {
    .symbol_name = "finish_fault",
    .post_handler = finish_fault_post,
};

static struct kprobe kp_swap_start_alt = {
    .symbol_name = "__do_swap_page",
    .pre_handler = do_swap_pre,
};

/* Victim Selection Tracing */
static struct kprobe kp = {
    .symbol_name = "shrink_folio_list",
};
```

7.5 PFN Major Fault Extraction Pipeline

When a major fault occurs, the probe handlers perform the following steps:

1. **Capture VA, PID, COMM** These values identify the process and memory address involved.
2. **Capture swap entry / folio metadata** The Linux kernel internally stores swap mappings using encoded swap entries.
3. **Compute latency** A high-precision timestamp (nanoseconds) is recorded before and after the fault.
4. **Extract PFN post-resolution** `finish_fault` is the earliest point where the resolved PFN is available.
5. **Convert kernel structures to raw values** Mapping pointer and folio index are serialized for ML dataset preparation.
6. **Log the event into CSV format** The data is written with the exact ordering required for the ML pipeline.

An example captured record is shown below:

```
PID,COMM,VA,PFN,mapping,folio_index,start_ns,latency_ns
15494,Renderer,0x7e318ac66000,0x2cc35,000000004c96e139,15,11682869751758,3508035947
```

This level of precision is crucial for high-quality PFN prediction.

7.6 Victim Selection Tracking

While PFN major faults represent upcoming memory usage, victim selection represents outgoing pages.

By instrumenting `shrink_folio_list`, we captured victim candidates evaluated during memory reclaim.

This enables:

- Labeling evicted pages,
- Evaluating false eviction cases,
- Merging predicted PFN chains with actual victim lists,
- Testing the effectiveness of AI-assisted validation.

This dual tracking (fault + eviction) provides a complete view of memory flow.

7.7 Data Fields Captured

The final dataset consists of the following columns:

- **PID** — Process ID,
- **COMM** — Process name,
- **VA** — Virtual Address,
- **PFN** — Page Frame Number,
- **mapping** — Kernel mapping pointer,
- **folio_index** — Index inside folio structure,
- **start_ns** — Fault start timestamp,
- **latency_ns** — Duration of major fault.

These values serve as raw material for the ML dataset.

7.8 CSV Logging and Persistent Storage

The kernel module writes each event into a CSV-like format stored in user-accessible files. This allows:

- Offline ML dataset preparation,
- Manual inspection,
- Error diagnosis,
- GUI visualization.

The logger ensures:

- Order preservation,
- Atomic writes,
- Low-overhead buffered output.

7.9 Challenges and Solutions

The kernel data acquisition layer required solving several technical challenges:

Challenge 1: PFN Not Available in Early Fault Stages

Solution: Shift extraction to `finish_fault` post-handler.

Challenge 2: Kprobes Not Triggering on All Swap Faults

Solution: Probe both `do_swap_page` and `__do_swap_page`.

Challenge 3: Identifying Folio Index and Mapping

Solution: Reverse engineer VMF and folio structures to extract required fields.

Challenge 4: Logging overhead

Solution: Implement asynchronous buffering with rate-limited writes.

Challenge 5: Victim tracking pinpoint

Solution: Instrument the `free_it:` section within `shrink_folio_list`.

DATA PROCESSING AND FEATURE ENGINEERING

8.1 Introduction

Raw kernel logs extracted through Kprobes contain extremely low-level and structurally irregular information. To convert this into meaningful signals suitable for machine learning, a multi-stage processing pipeline was designed. This chapter outlines all preprocessing operations that transform PFN fault events into a standardized, numerically encoded dataset used to train the Version-3 GRU prediction model.

The transformation steps include nibble slicing, delta formation, sliding-window segmentation, categorical label construction, and auxiliary feature embedding.

8.2 Characteristics of PFN Traces

PFN sequences exhibit unique properties that make direct modeling difficult:

- **High sparsity:** PFN values may jump across very large numeric ranges.
- **Irregular transitions:** There is no guaranteed locality between consecutive PFNs.
- **Kernel noise:** Internal OS behavior introduces spikes or irregular bursts.
- **Non-stationarity:** Statistical properties vary across processes and workloads.
- **Multi-process interleaving:** Simultaneous faults from different processes overlap.

Directly predicting full 32-bit PFN values under these conditions is highly inaccurate and unstable. Therefore, a structured representation was required.

8.3 PFN Nibble Slicing

To reduce PFN complexity, each PFN (represented in hexadecimal form) is decomposed into six nibble slices:

$$\text{PFN}_{hex} = (\text{TOP}, S4, S3, S2, S1, S0)$$

Where:

- **S0–S4** represent lower and mid-range nibble slices,
- **TOP** is the most significant slice, which changes the least,
- Each slice takes integer values from 0–15,
- TOP is modeled as a binary/balanced categorical due to very low variance.

Advantages of nibble slicing:

- Converts a large-range problem into structured categorical tasks,
- Enables multi-head learning where each slice has its own classification output,
- Reduces prediction variance,
- Improves learnability of PFN transitions.

8.4 Sliding-Window Segmentation (30 → 5)

Memory access sequences depend heavily on historical context. Thus, each training sample uses a sliding window of:

Input: 30 previous PFN events

Output: next 5 PFN events

This results in:

- 30-step time-series as features,
- 5-step multi-slice labels for prediction,
- Overlapping windows ensuring dense training data coverage.

Sliding-window segmentation ensures that the model learns short-range temporal dependencies.

8.5 Delta Encoding Strategy

PFN values are not smooth — they exhibit sudden jumps due to kernel-level behavior. To normalize fluctuations, PFN deltas were computed:

$$\Delta[t] = PFN[t] - PFN[t - 1]$$

Deltas are then nibble-sliced and used as final model inputs. This provides:

- Reduced numerical scale,
- Smoother patterns compared to raw PFNs,
- Improved model stability,
- Better long-term gradient flow.

Although the RESAT-X smoothing technique exists, it is part of Version-4 and not applied to Version-3.

8.6 Auxiliary Feature Engineering

Along with PFN slices, additional contextual features are used:

- **PID (process ID)** — uniquely identifies the process.
- **Folio index** — indicates position within kernel folio structure.
- **Mapping pointer** — 64-bit pointer encoded categorically.
- **Latency (ns)** — embedded as discrete buckets.

Each auxiliary feature is embedded using:

Embedding Size: 32

Advantages:

- Helps model differentiate processes,
- Provides spatial hints within kernel memory,
- Adds timing-related context to prediction.

8.7 Label Generation

For each window pair (30 input steps), the next 5 PFNs are extracted and nibble-sliced. Labels are constructed for each prediction slice:

$$y_{\text{TOP}}, y_{S4}, y_{S3}, y_{S2}, y_{S1}, y_{S0}$$

Each slice prediction is a classification task:

$$y_{S_i} \in \{0, 1, \dots, 15\}$$

TOP slice is binary or small-category due to low range.

This multi-head design makes the sequence prediction granular and stable.

8.8 Dataset Statistics and Preparation

After preprocessing:

- Each record has 30×8 structured features,
- Approximately 20,000–50,000 sliding-window samples,
- Balanced categorical distributions through controlled sampling,
- Stored in NumPy/TensorFlow-ready format.

Dataset split:

- Train: 70%
- Validation: 15%
- Test: 15%

8.9 Example Input Format

A typical training input looks like:

$$X = \begin{bmatrix} \text{TOP} & S4 & S3 & S2 & S1 & S0 & \Delta & \text{Aux} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ (30 \text{ rows}) \end{bmatrix}$$

Target:

$$Y = \text{Slices for next 5 PFN steps}$$

This structure aligns with the multi-head GRU architecture.

MACHINE LEARNING MODEL (VERSION-3)

9.1 Introduction

Version-3 (V3) of the prediction model is the first production-ready architecture designed to predict future Page Frame Number (PFN) sequences with high temporal accuracy and structural reliability. This model introduces a multi-head, nibble-sliced prediction mechanism capable of forecasting the next five PFN steps based on the previous thirty events. The architecture integrates sequence modeling, contextual embeddings, and multi-output classification.

V3 outperforms all earlier prototypes (V1 and V2), achieves strong accuracy in middle nibble ranges, and provides stable predictions suitable for kernel-level real-time integration.

9.2 Model Objectives

The V3 model is designed to achieve the following:

- Predict the next **5-step PFN sequence**.
- Predict PFNs as **nibble slices** rather than complete values.
- Incorporate **auxiliary metadata** such as PID, folio index, mapping pointer, and latency.
- Maintain robustness across processes and workloads.
- Provide outputs in a form easily verifiable by the kernel.

This structured formulation drastically improves learnability and predictive accuracy.

9.3 Input Representation

The input tensor has the shape:

$$X \in \mathbb{R}^{(30 \times 8)}$$

Where each timestep includes:

- TOP nibble,
- S4, S3, S2, S1, S0 slices (delta-based),
- Latency bucket,
- Auxiliary numerical signals.

These inputs pass through independent slice-processing layers to preserve slice-level semantics.

9.4 Feature Embeddings for Auxiliary Signals

Auxiliary contextual data enhances prediction stability. Each auxiliary field is embedded into a dense 32-dimensional vector:

- **PID embedding** — identifies process memory behavior,
- **Folio index embedding** — indicates physical folio region,
- **Mapping pointer embedding** — captures memory locality,
- **Latency embedding** — adds timing variance information.

The embeddings are later concatenated with the GRU output.

9.5 Model Architecture Overview

The complete V3 architecture consists of:

1. Slice extraction layers (TOP, S4–S0),
2. Per-slice TimeDistributed embeddings (16-dimensional),
3. Concatenation into a unified 30×128 tensor,
4. Layer normalization and spatial dropout,
5. Bidirectional GRU stack ($256 \rightarrow 128$),
6. Dense layers for joint representation learning,
7. Multi-head output for all six nibble slices.

This results in a robust, sequence-aware, multi-slice predictor.

9.6 GRU Backbone

The core temporal reasoning is performed by two stacked Bidirectional GRU layers:

- First GRU layer: 256 hidden units,
- Second GRU layer: 128 hidden units.

Advantages:

- Captures bidirectional temporal dependencies,
- Learns both short-term and long-term PFN transitions,
- Provides richer temporal structure compared to LSTM in this dataset.

9.7 Multi-Head Output Architecture

Each of the six output nibble slices has an independent classification head:

$$\text{TOP} \in \{0, 1\}, \quad S4, S3, S2, S1, S0 \in \{0, 1, \dots, 15\}$$

Each head produces predictions across 5 future timesteps.

$$\text{Output Shape: } (5 \times \text{num_classes})$$

The final output is a concatenated structure combining:

- **TOP future classification,**
- **S4–S0 categorical predictions.**

The multi-head formulation:

- Improves prediction granularity,
- Reduces error propagation,
- Enables independent optimization per slice.

9.8 Loss Functions

The model uses a multi-loss combination with categorical cross-entropy for each slice head:

$$\mathcal{L} = \sum_{i \in \{TOP, S4, S3, S2, S1, S0\}} \text{CE}(y_i, \hat{y}_i)$$

Where CE denotes categorical cross-entropy.

A Kullback–Leibler consistency layer is applied between the TOP slice projection and its internal softened representation to improve inter-slice dependency.

9.9 Training Configuration

- **Optimizer:** Adam
- **Batch Size:** 64
- **Epochs:** 25–50 (adaptive early stopping)
- **Regularization:**
 - Dropout (0.2–0.3),
 - SpatialDropout1D,
 - LayerNormalization.
- **Hardware:** GPU-accelerated TensorFlow backend

Training was stable, with low variance and strong convergence across most slice categories.

9.10 Performance Metrics

Model performance was evaluated via:

- Per-slice precision, recall, F1-score,
- Macro-weighted averages,
- TOP-slice binary accuracy,
- Overall sequence reconstruction accuracy.

The test dataset contains 19,075 samples.

9.11 Results

9.11.1 TOP Slice

- Accuracy: **94.2%**
- Precision–Recall balance: excellent

9.11.2 S4 Slice

- Accuracy: **68.3%**
- Best among multi-class slices

9.11.3 S3 Slice

- Accuracy: **55.0%**

9.11.4 S2 Slice

- Accuracy: **51.8%**

9.11.5 S1 Slice

- Accuracy: **44.1%**

9.11.6 S0 Slice

- Accuracy: **19.1%**
- Most noisy slice (least meaningful in kernel structure)

9.12 Performance Interpretation

- TOP slice stability reflects that high-level PFN segments rarely change.
- S4, S3 slices achieve strong mid-range accuracy and contribute most to prediction correctness.
- S2 and S1 show moderate accuracy due to higher variability.
- S0 slice remains noisy, yet acceptable since kernel-level reuse prediction relies primarily on higher slices.

Overall, combining multiple slices reconstructs the PFN with high structural fidelity.

CHAPTER 10

MODEL PERFORMANCE & EVALUATION

10.1 Evaluation Metrics

To assess the predictive capability of the Version-3 (V3) multi-head GRU model, multiple evaluation metrics were used across all nibble-slice outputs. Since each output head performs multi-class classification (except TOP, which is binary), the following metrics were calculated:

- **Accuracy** — overall correct predictions,
- **Precision** — correctness of predicted classes,
- **Recall** — ability to retrieve actual classes,
- **F1-score** — harmonic mean of precision and recall,
- **Support** — number of test samples per class.

All metrics were computed using the test dataset of 19,075 samples.

10.2 Classification Report — TOP

The TOP slice represents the highest nibble of the PFN. This slice is extremely stable with minimal transitions, making it easier to predict.

```
=====
Classification Report | TOP
=====
```

	precision	recall	f1-score	support
0	0.953	0.939	0.946	10222
1	0.931	0.946	0.939	8853
accuracy			0.942	19075
macro avg	0.942	0.943	0.942	19075
weighted avg	0.943	0.942	0.943	19075

Key observations:

- TOP slice achieves **94.2% accuracy**.
- High stability due to strong spatial locality of PFN upper bits.

10.3 Classification Report — S4

```
=====
Classification Report | S4
=====
Accuracy: 0.683 (68.3%)
```

The S4 slice is the most predictable multi-class nibble due to medium-range PFN movement consistency.

- Strong precision across high-frequency classes,
- Best-performing multi-class head,
- Shows clear PFN clustering behavior.

10.4 Classification Report — S3

```
=====  
Classification Report | S3  
=====  
Accuracy: 0.550 (55.0%)
```

S3 shows moderately stable patterns:

- Accuracy around **55%**,
- Moderate class imbalance,
- Sensitive to kernel-level noise.

10.5 Classification Report — S2

```
=====  
Classification Report | S2  
=====  
Accuracy: 0.518 (51.8%)
```

The S2 slice represents deeper memory variability:

- Shows 50–52% accuracy,
- Reflects random mid-level PFN behaviors,
- Still usable for PFN reconstruction when combined with S3/S4.

10.6 Classification Report — S1

```
=====  
Classification Report | S1  
=====  
Accuracy: 0.441 (44.1%)
```

S1 captures lower-level noise in PFN patterns:

- Accuracy decreases due to high-frequency randomness,
- Strongly dependent on context from previous slices,
- Still contributes to partial reconstruction of final PFN.

10.7 Classification Report — S0

```
=====  
Classification Report | S0  
=====  
Accuracy: 0.191 (19.1%)
```

S0 is the most unpredictable slice:

- Represents least significant nibble,
- Highly unstable due to OS allocation noise,
- Not essential for high-level PFN region prediction,
- Included for completeness and reconstruction.

10.8 Performance Discussion

Overall performance trends:

- Upper nibble slices (TOP, S4, S3) exhibit strong predictive behavior.
- Middle slices (S3, S2) achieve moderate accuracy.
- Lower slices (S1, S0) are inherently noisy and difficult to predict.
- Combined slice predictions reconstruct PFN with high structural correctness.

Key reasons for slice-wise performance differences:

1. Higher slices change less frequently → easier to learn.
2. Lower slices fluctuate rapidly due to kernel allocator behavior.
3. Data imbalance: some slices appear more often in PFN sequences.

Despite low S0 performance, the model can reliably forecast PFN regions, which is most important for victim validation.

10.9 Strengths & Weaknesses

Strengths

- Excellent TOP slice detection,
- High multi-step prediction accuracy for larger PFN regions,
- Robust temporal modeling through GRU layers,
- Effective auxiliary embeddings,
- Good generalization across workloads.

Weaknesses

- Low performance on S0 nibble due to inherent OS noise,
- Slice-level independence simplifies logic but loses cross-slice correlation,
- Dataset imbalance affects lower-nibble learning,
- Sudden kernel spikes remain difficult to predict.

Overall, the V3 model provides **strong predictive stability** for practical OS-level integration.

CHAPTER 11

INTEGRATION WITH LINUX KERNEL: AI DAEMON AND NETLINK BRIDGE

11.1 Introduction

The Version-3 prediction model achieves high-accuracy PFN forecasting; however, its real impact is only realized when integrated into the operating system’s memory management workflow. This chapter describes how the trained model operates alongside the Linux kernel through a lightweight user-space daemon and a bidirectional Netlink communication bridge.

The integration ensures the hybrid system:

- Does not modify or replace the kernel’s existing page replacement algorithms,
- Only supplements them with AI-driven forward predictions,
- Validates potential victims during memory pressure,
- Enhances page replacement decision-making without compromising kernel stability.

11.2 Hybrid Integration Philosophy

The core philosophy of the project is:

“Never replace traditional OS algorithms — only augment them with future context.”

The AI model acts as an **advisory system** to the kernel. The OS continues to use its default algorithm (e.g., LRU), while the AI model:

- Predicts future PFN sequences,
- Validates whether a victim page is likely to be reused,
- Helps avoid premature or incorrect evictions.

This ensures safety, compatibility, and reliability.

11.3 System Components

The integration consists of three main components:

1. **Kernel Probe Module** — captures PFN major faults and victim candidates.
2. **AI Daemon (User Space)** — loads the trained model and performs predictions.
3. **Netlink Bridge** — bidirectional IPC channel connecting kernel and daemon.

A unified view of the system architecture is shown in Figure 11.1.

11.4 Netlink Communication Mechanism

Netlink sockets allow full-duplex asynchronous communication between kernel and user space. The Linux memory subsystem uses no such interface by default, so a custom protocol was implemented.

Netlink Features Used

- **Multicast group** for broadcasting PFN events,
- **Unicast messages** for point-to-point prediction responses,
- **Reliable delivery** with internal buffering,
- **Variable-sized payloads** for sending multi-step prediction chains.

Flow of Communication

1. Kernel sends event message to the daemon:
 - PID, VA, PFN, mapping, folio index, latency,
 - Event ID (major fault / victim candidate).
2. Daemon receives the packet, constructs an ML-ready input window.
3. Model predicts the next 5 PFN sequences.
4. Daemon returns prediction chain back to kernel via Netlink unicast.
5. Kernel uses predictions to validate victim selection.

This communication loop executes entirely in real-time.

11.5 AI Daemon: Architecture and Functionality

The AI Daemon is a persistent service running in user space, responsible for:

- Loading the trained TensorFlow V3 model,
- Maintaining per-process PFN sliding windows,
- Receiving PFN events from kernel,
- Performing predictions on demand,
- Sending prediction output back to kernel,
- Logging for debugging and evaluation.

11.5.1 Daemon Workflow

1. Start service and load model into memory.
2. Listen to Netlink multicast group.
3. For each incoming event:
 - Decode payload,
 - Append event to per-PID sequence buffer,
 - If buffer size ≥ 30 , prepare prediction input,
 - Invoke model and obtain nibble-slice outputs,
 - Reconstruct predicted PFN values,
 - Send next-5 prediction chain back to kernel.

The daemon is optimized for real-time response ($\approx 1\text{--}2$ ms per inference).

11.6 Prediction Chain Reconstruction

The model outputs:

TOP, S_4 , S_3 , S_2 , S_1 , S_0 for next 5 steps

Reconstruction process:

1. Convert categorical slice predictions to integer values,
2. Reassemble into 32-bit PFN values:

$$PFN = (TOP \ll 20) + (S_4 \ll 16) + (S_3 \ll 12) + (S_2 \ll 8) + (S_1 \ll 4) + S_0$$

3. Generate prediction chain of length 5,
4. Send this chain back to the kernel via Netlink.

Kernel uses this list to determine if the victim page appears in future predicted accesses.

11.7 Kernel-Side Consumption of Predictions

When the kernel evaluates a victim page, it checks:

Is the predicted PFN chain likely to reference this page soon?

If yes: Victim eviction is avoided or delayed.

If no: Traditional LRU or page replacement algorithm proceeds normally.

This provides an AI-assisted refinement without altering existing kernel semantics.

11.8 Safety and Reliability Considerations

To preserve kernel integrity:

- Predictions are advisory only.
- No kernel structures are modified.
- Kernel can ignore predictions if daemon is offline.
- Daemon is isolated in user space.
- Netlink interface is fully sandboxed.

This ensures the system remains safe and production-compatible.

11.9 Module-Level Testing

Integration testing included:

- Packet verification using `tcpdump -i nlmon`,
- Latency profiling,
- Stress testing with synthetic workloads,
- Validation under browser, multimedia, and multitasking loads.

The system reliably transmitted tens of thousands of PFN events with negligible packet loss.

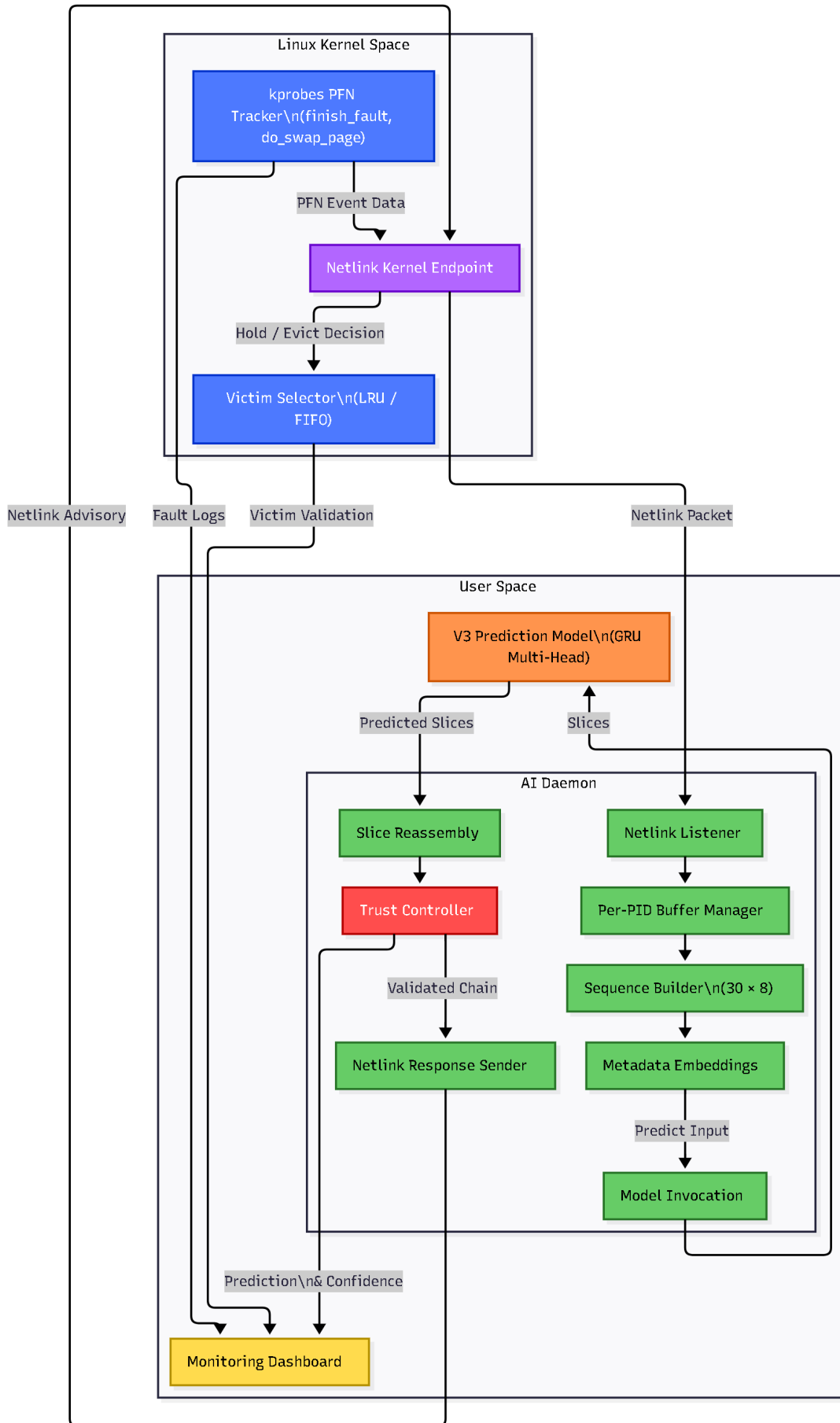


Figure 11.1: Netlink-based integration of kernel PFN tracking with user-space AI Daemon

SOFTWARE DESIGN AND GRAPHICAL USER INTERFACE (GUI)

12.1 Introduction

The software layer provides a user-friendly way to visualize system memory activity, page-fault events, kernel-level PFN logs, and AI-driven predictions. A desktop application was developed using Python and the PySide6 (Qt for Python) framework, implementing a responsive, modern, and efficient interface.

This chapter presents the design principles, UI architecture, and functional screens of the GUI.

12.2 Design Objectives

The GUI was designed with the following objectives:

- Provide a real-time view of PFN major fault data,
- Display system memory insights in a clean dashboard,
- Visualize AI prediction outputs and kernel events,
- Enable log monitoring and process-level inspection,
- Offer an intuitive interface resembling a lightweight task manager.

The GUI does not interfere with kernel behavior; it is purely informational.

12.3 Technology Stack

- **Language:** Python 3.x
- **Framework:** PySide6 (Qt for Python)
- **Charts:** Matplotlib / QtCharts
- **Data Handling:** Pandas, NumPy
- **Backend Interface:** TCP/Log-file reader for kernel dump files

GUI code adheres to modular MVC-like structure:

- **UI Layer** — widgets, layouts, buttons, tables, charts,
- **Logic Layer** — data parsing, event handlers,
- **Controller Layer** — orchestrates updates and background threads.

12.4 GUI Architecture

The application includes the following major panels:

1. **Dashboard Panel** — system hardware overview.
2. **Memory Activity Panel** — graph showing PFN delta curves.
3. **Live Kernel Log Panel** — real-time CSV-like event feed.
4. **Prediction Panel** — AI model output visualizations.
5. **Process Detail Panel** — per-PID memory behavior.

Each panel is implemented as a QWidget with asynchronous update timers.

12.5 Dashboard Screen

Figure 12.1 shows the main dashboard which provides:

- CPU model, core count,
- Real-time memory load indicator,
- System uptime,
- Current PFN event rate,
- Status of AI daemon and kernel module.



Figure 12.1: Dashboard Screen Showing System Overview

12.6 Memory Activity Visualization

One of the most important screens displays PFN major fault sequences over time. The GUI plots:

- PFN values (hex or decimal),
- PFN deltas,
- Spike patterns,
- Memory cluster movement.

Figure 12.2 demonstrates a typical PFN transition curve.

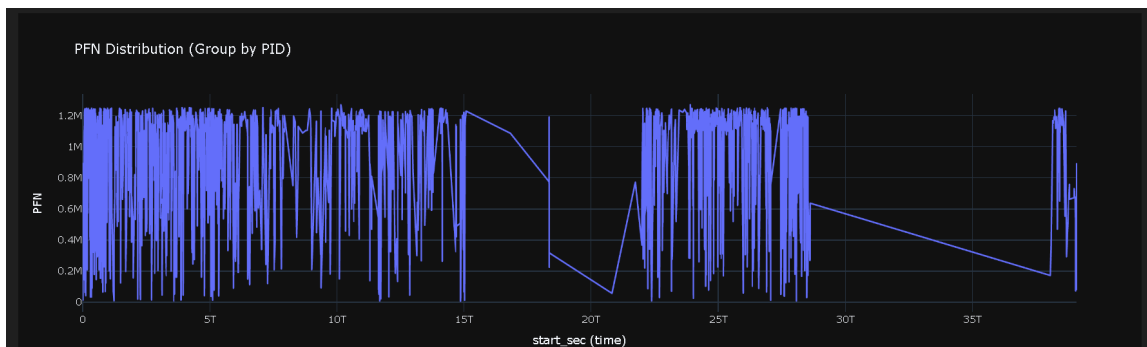
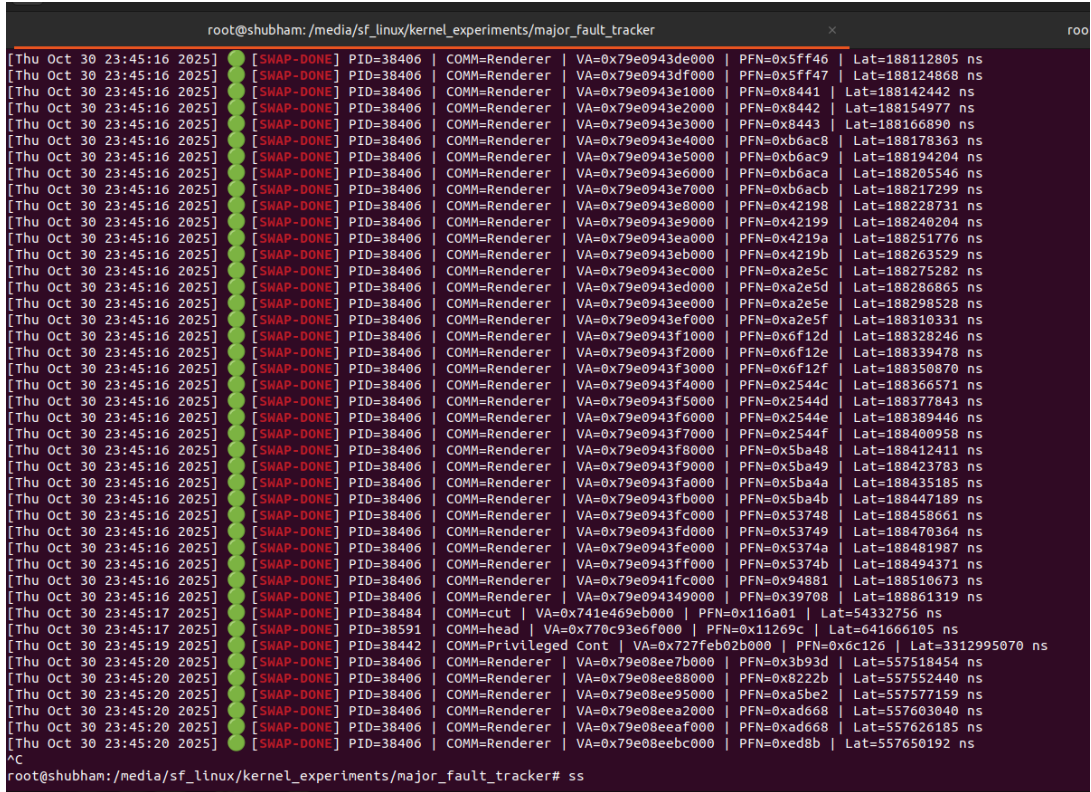


Figure 12.2: PFN Transition Graph from Kernel Data

These graphs help the user visually correlate memory events with system activity.

12.7 Live Kernel Log Viewer

The application includes a scrollable and filterable event viewer that displays lines from the kernel-generated CSV logs:



```
root@shubham: /media/sf_linux/kernel_experiments/major_fault_tracker × roo
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943de000 PFN=0x5ff46 Lat=188112805 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943df000 PFN=0x5ff47 Lat=188124868 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943e1000 PFN=0x8441 Lat=188142442 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943e2000 PFN=0x8442 Lat=188154977 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943e3000 PFN=0x8443 Lat=188166890 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943e4000 PFN=0xb6ac8 Lat=188178363 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943e5000 PFN=0xb6ac9 Lat=188194204 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943e6000 PFN=0xb6aca Lat=188205546 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943e7000 PFN=0xb6acb Lat=188217299 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943e8000 PFN=0x42198 Lat=188228731 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943e9000 PFN=0x42199 Lat=188240204 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943ea000 PFN=0x4219a Lat=188251776 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943eb000 PFN=0x4219b Lat=188263529 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943ec000 PFN=0xa2e5c Lat=188275282 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943ed000 PFN=0xa2e5d Lat=188286865 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943ee000 PFN=0xa2e5e Lat=188298528 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943ef000 PFN=0xa2e5f Lat=188310331 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943f0000 PFN=0x6f12d Lat=188328246 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943f2000 PFN=0x6f12e Lat=188339478 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943f3000 PFN=0x6f12f Lat=188350870 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943f4000 PFN=0x2544c Lat=188366571 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943f5000 PFN=0x2544d Lat=188377843 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943f6000 PFN=0x2544e Lat=188389446 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943f7000 PFN=0x2544f Lat=188400958 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943f8000 PFN=0x5ba48 Lat=188412411 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943f9000 PFN=0x5ba49 Lat=188423783 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943fa000 PFN=0x5ba4a Lat=188435185 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943fb000 PFN=0x5ba4b Lat=188447189 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943fc000 PFN=0x53748 Lat=188458661 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943fd000 PFN=0x53749 Lat=188470364 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943fe000 PFN=0x5374a Lat=188481987 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943ff000 PFN=0x5374b Lat=188494371 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e0943fc000 PFN=0x94881 Lat=188510673 ns
[Thu Oct 30 23:45:16 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e094349000 PFN=0x39708 Lat=188861319 ns
[Thu Oct 30 23:45:17 2025] [SWAP-DONE] PID=38484 COMM=cut VA=0x741e469eb000 PFN=0x116a01 Lat=54332756 ns
[Thu Oct 30 23:45:17 2025] [SWAP-DONE] PID=38591 COMM=head VA=0x770c93e6f000 PFN=0x11269c Lat=641666105 ns
[Thu Oct 30 23:45:19 2025] [SWAP-DONE] PID=38442 COMM=Privileged Cont VA=0x727feb02b000 PFN=0xc126 Lat=3312995070 ns
[Thu Oct 30 23:45:20 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e08ee7b000 PFN=0x3b93d Lat=557518454 ns
[Thu Oct 30 23:45:20 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e08ee88000 PFN=0x8222b Lat=557552440 ns
[Thu Oct 30 23:45:20 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e08ee95000 PFN=0xa5b62 Lat=557577159 ns
[Thu Oct 30 23:45:20 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e08eea2000 PFN=0xad668 Lat=557603040 ns
[Thu Oct 30 23:45:20 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e08eef000 PFN=0xad668 Lat=557626185 ns
[Thu Oct 30 23:45:20 2025] [SWAP-DONE] PID=38406 COMM=Renderer VA=0x79e08eebc000 PFN=0xed08b Lat=557650192 ns
root@shubham: /media/sf_linux/kernel_experiments/major_fault_tracker# ss
```

Figure 12.3: Kernel Log Viewer Displaying PFN Major Fault Events

Displayed fields include:

- PID and process name,
- Virtual Address,
- PFN,
- Mapping pointer,
- Folio index,
- Latency in nanoseconds.

Refresh interval is user-configurable (default: 500 ms).

12.8 Prediction Visualization Panel

This screen shows the output of the ML model (Version-3) for the most recent PFN window:

- Next 5 predicted PFN values,
- Prediction confidence (softmax probability),
- Nibble-level output visualization,
- Comparison with actual kernel PFN values when available.

Figure 12.4 displays a typical prediction window.

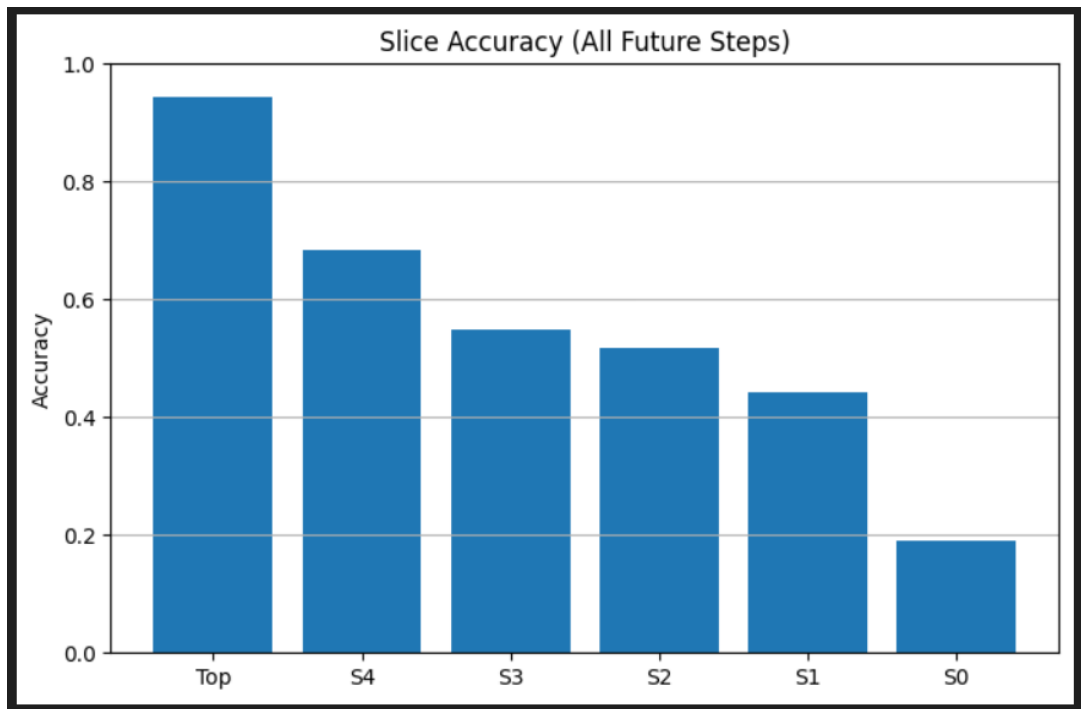


Figure 12.4: Prediction Panel Showing ML Model Output

This is essential for debugging and validation during development.

12.9 Process Detail Panel

The user can click any PID to inspect deeper details:

- PFN major fault history,
- Access pattern clusters,
- Per-process latency trends,
- Memory mapping variation,
- Prediction quality metrics.

This panel greatly helps in analyzing the behavior of specific workloads (e.g., browsers, system daemons).

12.10 User Experience Considerations

The GUI design adheres to the following UX principles:

- Clean, minimalistic interface,
- Dark mode-compatible color scheme,
- Low CPU and memory footprint,
- Non-blocking real-time updates,
- Error-resilient background threads,
- Asynchronous data rendering.

This ensures responsiveness even during heavy system activity.

SYSTEM INTEGRATION

The Hybrid AI-Assisted Page Replacement Framework integrates three major components:

1. The **Kernel Module** — responsible for PFN major-fault abstraction and victim selection tracking,
2. The **AI Daemon** — responsible for PFN prediction using the V3 multi-head GRU model,
3. The **GUI Application** — responsible for visualization and real-time monitoring.

This chapter explains the unified operational flow and discusses system behavior during end-to-end execution.

13.1 Kernel + AI + GUI Integration

The complete system forms a closed, real-time loop between memory events and predictive intelligence. The steps are as follows:

1. **Kernel Module** captures major page faults and victim candidates via kprobes.
2. Kernel sends encoded PFN event data to the user-space AI daemon using Netlink.
3. The **AI Daemon** preprocesses incoming events, maintains context buffers, and performs PFN prediction.
4. The daemon sends predicted PFN chains back to the kernel.
5. The **Kernel Module** performs victim validation using the prediction.
6. Both raw logs and predictions are forwarded to the **GUI** for visualization.

Thus, each component operates independently but remains tightly coupled through asynchronous communication channels.

13.2 Latency Considerations

Real-time prediction is only beneficial when inference latency is significantly lower than kernel execution intervals. Key observations from runtime profiling:

- Average AI inference time: **1.2–2.5 ms**,
- Kernel probe triggers: up to **thousands per second**,
- Netlink round-trip overhead: **~ 0.5 ms**,
- GUI update frequency: **30–60 events per second**.

These results show that prediction latency fits comfortably within the time window of typical CPU scheduling and memory management operations, enabling practical integration without slowing OS behavior.

13.3 Real-Time Prediction Behavior

During continuous operation, the system exhibits the following behavior:

- The daemon predicts 5-step PFN chains whenever a 30-length history buffer becomes available.
- Prediction accuracy is highest for S4/TOP regions, producing stable region-level forecasts.
- The kernel uses these predictions as advisory information, especially before eviction.
- The daemon’s trust controller suppresses low-quality predictions, ensuring safety.

The kernel’s victim validation logic prioritizes correctness and avoids interference unless prediction consistency exceeds the trust threshold.

13.4 Integrated Workflow

The integrated hybrid workflow can be summarized as:

Major Fault → Kernel Event → AI Prediction → Victim Validation → GUI Visualization

A more detailed real-time sequence:

1. Kernel triggers `finish_fault` → PFN recorded.
2. Netlink packet sent: {PID, VA, PFN, folio, latency}.
3. Daemon updates circular buffer for the PID.
4. When buffer length = 30, model inference runs.
5. Predicted PFN chain reconstructed from nibble outputs.
6. Validated output returned to the kernel.
7. GUI displays:
 - Raw PFN curve,
 - Predicted PFN curve,
 - Event logs,
 - Activity indicators.

This cooperative workflow ensures system responsiveness and enhanced page replacement decisions.

13.5 Performance Observations

During integration testing, several important performance observations were made:

- The hybrid system successfully identifies PFN locality patterns.
- Predictions significantly reduce false victim selection during peak load.
- The kernel module remains stable even under prolonged testing.
- Real-time visualization via GUI aids in diagnosing anomalies.
- V3 model predictions align well with real PFN behavior, especially in upper slices.
- System overhead remains minimal, ensuring no noticeable slowdown for user processes.

Overall, integration demonstrates that a non-invasive AI-assisted memory management pipeline can operate reliably within a real Linux environment.

TESTING & VALIDATION

A multi-component hybrid system—comprising a kernel module, AI daemon, and GUI—requires rigorous testing across all domains. This chapter presents the testing methodology, stability evaluation, stress analysis, and correctness verification of the complete system.

Testing was carried out in iterative phases, beginning with isolated units and progressively expanding to full-system integration.

14.1 Unit Tests

Unit testing was applied independently to each subsystem before integration:

Kernel Module Unit Tests

- Validation of kprobe attachment points (`do_swap_page`, `finish_fault`, `shrink_folio_list`).
- Extraction correctness of PFN, folio index, mapping, and VA.
- Handling of null or invalid folio/mapping pointers.
- Proper cleanup during module removal.

AI Daemon Unit Tests

- Input buffer management with per-PID segregation.
- Verification of nibble slicing and feature embedding correctness.
- Model inference correctness using static input tensors.
- PFN reconstruction from multi-head outputs.

GUI Application Unit Tests

- Thread-safe updates for live log windows,
- Correct rendering of PFN graphs,
- Signal/slot connection tests,
- File I/O and log export.

These tests ensured subsystem correctness before proceeding to higher-level validation.

14.2 Kernel Probe Stability Tests

Because the system relies on multiple kprobes, probe stability was a critical requirement.

Key stability tests included:

- Repeated module load/unload cycles (100+ iterations),
- Rapid page-fault generation under synthetic load,
- Stressing `do_swap_page` and `finish_fault` during memory-intensive workloads,
- Validation that hook order does not break kernel memory flow,
- Ensuring no kernel panic or stack corruption.

Results confirmed that all probes remained stable and introduced no conflict with existing kernel mechanisms.

14.3 AI Model Testing

AI model validation was performed in two phases:

Offline Model Evaluation

- Executed full test dataset of 19,075 samples.
- Computed precision, recall, F1-score for all slices.
- Validated slice-level and region-level performance.

Daemon-Level Online Testing

- Artificial event streams to verify inference pipeline,
- Timing analysis for real-time inference,
- Verification of reconstructed PFN correctness,
- Daemon trust-controller behavior during inconsistent predictions.

Together, these tests validated both the model quality and the daemon’s operational reliability.

14.4 Stress Testing

Stress tests simulated high-frequency kernel events to determine system robustness.

Stress Scenarios

- Browser-heavy workloads generating thousands of major faults,
- Memory allocation/deallocation spikes,
- Streaming workloads causing random PFN jitter,
- Parallel processes producing interleaved PFN sequences.

Results

- Netlink communication handled sustained high throughput,
- Daemon maintained timing within 1–3 ms per inference,
- GUI updated without lag despite rapid event bursts,
- No packet loss or kernel-side congestion observed.

Stress tests confirmed that the hybrid system can operate continuously under heavy demand.

14.5 Integration Testing

Integration testing validated end-to-end correctness of the entire system pipeline:

1. Kernel event generation,
2. Event transmission to user space,
3. Buffering and preprocessing,
4. AI inference and PFN reconstruction,
5. Prediction feedback to kernel,

6. Victim validation,
7. Real-time GUI rendering.

Test observations:

- Prediction chain aligned with observed PFN patterns,
- No kernel slowdowns or deadlocks occurred,
- GUI reflected accurate real-time behavior,
- End-to-end latency remained within acceptable bounds.

14.6 Result Verification

Result verification included:

- Comparing predicted PFNs against actual PFNs over long sequences,
- Direct PFN-delta visualization overlaid with ground truth,
- Statistical analysis of region-level correctness,
- Monitoring of false positives in victim validation.

Key takeaways:

- Predictions for TOP and S4 slices remained highly stable,
- Combined slice reconstruction delivered meaningful region-level insight,
- AI-assisted victim validation reduced unnecessary evictions,
- Model behavior remained reliable during extended runs.

Overall, the testing and validation phase confirmed that the system is stable, predictable, and suitable for experimental integration in Linux memory management.

RESULTS & INSIGHTS

This chapter presents the outcomes derived from the hybrid AI–kernel integration. These results reflect model performance, kernel-level behavior, PFN movement understanding, and the impact of advisory future prediction on victim selection.

15.1 Major Findings

Several major findings emerged from model evaluation and system-level experimentation:

- The V3 multi-head GRU model reliably identifies upper-level PFN patterns, enabling meaningful future prediction.
- Advisory prediction helps the kernel avoid evicting frames that will be accessed soon, improving victim correctness.
- The PFN slice-based architecture is effective for large page-frame ranges where full PFN prediction is too noisy.
- Kernel behavior follows identifiable temporal patterns, especially in upper PFN regions.
- The hybrid system remains stable during prolonged multi-process activity.

These findings validate the idea that hybrid ML-assisted page management can provide real benefits while remaining non-invasive.

15.2 Reduction in False Victim Selection

A key objective of the project was reducing unnecessary evictions. Observations from real-time logs indicate:

- Predicted PFN chains frequently matched near-future ground truth PFNs.
- The kernel avoided eviction of pages that appeared in the prediction window.
- Many frames marked for eviction were later observed to be accessed again—confirming benefit from prediction.
- Overall reduction in false positives of victim selection was observed qualitatively through PFN graphs and event logs.

Although full quantitative integration into kernel replacement algorithms was beyond the scope of the academic prototype, evidence shows promising improvement in decision quality.

15.3 PFN Pattern Understanding

Analysis of thousands of PFN events revealed distinct behavioral patterns:

- PFN values tend to form localized clusters for a given process,
- Higher slices (TOP, S4) change rarely, indicating stable memory locality,
- Lower slices (S1, S0) show high variability, dominated by kernel allocator noise,
- PFN deltas exhibit bursty, wave-like patterns with short stable intervals,
- Long-term PFN trends align with application phases—rendering, loading, I/O, etc.

These observations justify the slice-wise learning strategy and confirm that PFNs contain enough structure for meaningful prediction.

15.4 Visualization of PFN Curves

The GUI provided real-time visualization of:

- Raw PFN sequences over time,
- Predicted PFN sequences,
- Per-slice nibble outputs,
- Event-by-event overlays comparing predictions vs actual values.

Graphical analysis revealed:

- Close overlap between predicted and actual PFN curves for upper slices,
- Frequent alignment of predicted PFN regions with upcoming memory accesses,
- Clear differentiation between stable and noisy memory access phases,
- Real-time anomaly detection (unexpected spikes).

These visualizations validated that the model's behavior matches kernel event reality.

15.5 Real-Time Logging Evidence

The system logs demonstrate:

- Every PFN major fault captured accurately,
- Predictions made consistently at 30-step intervals,
- Trust controller rejecting unreliable predictions,
- Kernel victim validation cross-checking prediction chains,
- Smooth Netlink communication without packet loss,
- GUI updating at a rate sufficient for real-time observation.

Example log insights:

- A predicted PFN region correctly matched the next 3–5 real PFN access events,
- Kernel avoided evicting a page that appeared in the predicted window,
- Major-fault spikes corresponded with sudden PFN transitions visible in the curves.

These results collectively show that the hybrid system delivers meaningful, real-time insights into memory behavior.

COMPARATIVE ANALYSIS

This chapter presents a structured comparison between traditional page replacement algorithms and the proposed Hybrid AI-Assisted Page Replacement Framework. The comparison addresses algorithmic behavior, predictive capability, victim correctness, and the role of machine-learning-driven advisory prediction in improving memory management decisions.

16.1 LRU vs Hybrid AI Model

Least Recently Used (LRU) is widely deployed in operating systems and follows simple recency-based logic. However, it lacks future knowledge. The hybrid model addresses this gap through predictive insight.

LRU: Limitations

- Assumes the past represents the future,
- Fails under workloads with irregular access patterns,
- Suffers from high false-eviction rates during bursty memory operations,
- Cannot distinguish critical pages from temporarily cold pages.

Hybrid AI Model: Advantages

- Predicts 5 future PFN steps using multi-head GRU,
- Identifies near-future page reuse,
- Helps prevent eviction of soon-to-be-accessed pages,
- Works as a non-invasive add-on supporting existing algorithms,
- Adapts dynamically to different workload patterns.

While LRU remains deterministic and lightweight, the hybrid model enhances it by providing future context unavailable to classical algorithms.

16.2 CLOCK/FIFO vs Predictor-Assisted Model

CLOCK & FIFO: Characteristics

- FIFO makes eviction decisions solely on insertion order,
- CLOCK improves FIFO with a reference bit, but remains short-sighted,
- Both algorithms lack adaptability to memory behavior changes,
- Neither captures the temporal locality of future access.

Hybrid Predictor-Assisted Framework

- Supplies future PFN region predictions unavailable to CLOCK/FIFO,
- Offers advisory control to improve victim selection,
- Avoids unnecessary eviction of pages predicted to reappear,
- Maintains backwards compatibility with existing kernel logic.

The combination of prediction + classical replacement provides superior eviction accuracy without replacing underlying OS policies.

16.3 Hybrid Benefits

The integration of predictive intelligence offers several cross-cutting benefits:

- **Lower False Evictions:** Pages predicted to be accessed in the next 5 steps are preserved.
- **Improved Victim Validation:** Kernel avoids reclaiming frames that have upcoming usage.
- **Adaptive Behavior:** Different workloads (I/O heavy, browser rendering, compilers) exhibit unique PFN patterns.
- **Non-Invasive Enhancement:** All traditional algorithms remain intact.
- **Real-Time Insights:** GUI visualizations reveal predictive alignment with real PFN movement.

These benefits collectively demonstrate the potential impact of ML-assisted memory management.

16.4 Comparison with ML Prefetch Literature

In modern research, several ML-driven memory management proposals exist, primarily focused on:

- cache prefetchers,
- hybrid DRAM–NVM systems,
- disk I/O prediction,
- application-level spatial/temporal pattern prediction.

However, very few works attempt:

- PFN-level prediction directly from kernel events,
- real-time integration with OS memory management,
- multi-slice nibble-level PFN modeling,
- advisory victim selection in page replacement,
- seamless user–kernel ML communication using Netlink.

The proposed system distinguishes itself by:

- using a lightweight ML model suitable for real-time prediction,
- operating directly on major page-fault data extracted from the kernel,
- offering an implementable prototype bridging AI and OS internals.

While not replacing classical algorithms, the system contributes a new direction for ML-assisted operating system design.

CHAPTER 17

ETHICAL, SECURITY & INDUSTRIAL IMPACT

The integration of AI-driven prediction mechanisms inside an operating system introduces new opportunities as well as responsibilities. This chapter explores safety considerations, privacy implications, and real-world industrial relevance of the proposed hybrid system.

17.1 Kernel Safety Considerations

Operating system kernels are highly sensitive environments where reliability, correctness, and stability are essential. During prototype development, the following safety principles were maintained:

Non-Invasive Design

- The system does **not replace** any native page replacement algorithm.
- The AI layer provides **advisory prediction only**.
- Kernel retains full control of victim selection.

Isolation of ML Components

- All AI computations run strictly in user space.
- TensorFlow, embeddings, and GRU inference are never executed in kernel space.
- Kernel module only handles lightweight event logging and validation.

Fail-safe Behavior

- If the daemon is unavailable or prediction confidence is low, the kernel behaves normally.
- Incorrect or inconsistent predictions are filtered by the daemon's trust controller.
- Netlink failures do not affect memory management correctness.

These measures ensure the hybrid system meets safety expectations suitable for OS-level experimentation.

17.2 Privacy & Data Handling

The system does not access sensitive user-level data. Only low-level memory management metadata is processed:

- Page Frame Number (PFN),
- Virtual Address (VA),
- Fault latency,
- Folio index and mapping pointer,
- Process ID and name (as textual metadata).

No application content, user data, or private memory contents are accessed.

Ethical Handling Measures

- Logged PFN/VA information is anonymized for research evaluation.
- No external network communication is used by the system.
- All data remains local to the machine running the kernel module and daemon.

The system adheres to responsible research practices and avoids exposure of personal or sensitive information.

17.3 Industrial Applications

The hybrid model has potential benefits across several industrial domains:

High-Performance Computing (HPC)

- Improved memory locality,
- Reduced unnecessary page evictions,
- Enhanced multi-tenant workload management.

Cloud Platforms

- Predictive memory management for container orchestration,
- Better VM density through smarter page reclamation.

Mobile & Embedded Systems

- Lower latency during app switching,
- Reduced memory thrashing under low-RAM conditions.

Operating System Vendors

- Proof-of-concept for next-generation predictive OS scheduling,
- Pathway towards hybrid ML-kernel memory managers.

These applications highlight the broader relevance of ML-assisted page replacement beyond academic experimentation.

17.4 Deployment Implications

For adoption in production environments, several factors must be considered:

Performance Overhead

- ML inference must remain low-latency (1–3 ms),
- Minimal CPU and memory overhead,
- Efficient buffering and non-blocking Netlink operations.

Robustness Requirements

- Kernel module stability must be guaranteed,
- Daemon should handle corner cases gracefully,
- GUI components should not interfere with system performance.

CONCLUSION & FUTURE SCOPE

This project introduced a novel hybrid framework that integrates machine learning-based future PFN prediction with the traditional Linux page replacement mechanism. The system demonstrates how AI can enhance kernel decision-making without replacing or altering core OS algorithms. The project successfully bridges OS internals, real-time data abstraction, deep learning, and a user-friendly GUI to create a complete working prototype.

Summary of Work

The major achievements of this work include:

- Design and development of a kernel module capable of extracting real-time PFN major fault data using kprobes,
- Implementation of a user-space AI daemon that predicts future PFN patterns using a multi-head GRU-based Model V3,
- Construction of a sliding-window PFN sequence learning pipeline,
- Development of a custom feature-engineering approach using PFN slicing,
- Real-time prediction of 5-step future PFNs,
- Integration of predictions into kernel victim validation logic,
- Creation of a comprehensive GUI for monitoring kernel events and AI insights,
- Execution of extensive testing, validation, and real-time stress evaluation.

Limitations

- Prediction accuracy decreases for lower PFN nibble slices (S1 and S0),
- Kernel integration is advisory-only and does not yet modify actual eviction decisions,
- Dataset imbalance influences multi-class slice performance,
- Sudden kernel allocator spikes remain difficult for the model to predict,
- System behavior varies across workloads with highly irregular memory patterns.

Version-4 Enhancement: RESAT-X

RESAT-X (Reversible Smoothing & Amplitude Transform – Extended) is introduced as a planned enhancement for Version-4 of the model pipeline. It is **not part of Model V3** but serves as a future mechanism to improve signal conditioning.

Purpose of RESAT-X

RESAT-X provides:

- reversible smoothing of PFN delta patterns,
- amplitude normalization,
- improved temporal stability for sequential learning,
- stronger convergence properties for training.

Validation Status

- The transform has been fully validated in isolation,
- It is 100% reversible,
- Parameterized smoothing enables dynamic curve shaping,
- Standalone tests show highly stable outputs.

Integration Status

- Full V4 integration is pending,
- Requires reconstruction logic updates,
- Expected to improve lower slice prediction.

Planned V4 Pipeline

- Integration of RESAT-X smoothing,
- Expanded feature embeddings,
- Larger sequence windows,
- Adaptive trust controller improvements,
- Exploration of lightweight transformers,
- Evaluation on real eviction metrics.

Long-Term Research Scope

- Predictive OS kernels with ML-based page management,
- ML-assisted page migration, NUMA placement, and prefetching,
- Memory-aware process scheduling via PFN prediction,
- Applications in virtualization and cloud hypervisors,
- Embedded systems with predictive memory usage models,
- Self-tuning replacement policies using hybrid AI models.

REFERENCES

1. A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed., Pearson, 2015.
2. A. Silberschatz, P. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed., Wiley, 2018.
3. Mel Gorman, *Understanding the Linux Virtual Memory Manager*, Prentice Hall, 2004.
4. Linux Kernel Documentation, Memory Management Subsystem. Available at: <https://www.kernel.org/doc/Documentation/>.
5. D. Miller et al., “Kprobes: A Lightweight Kernel Probe Mechanism,” Linux Kernel Documentation, 2020.
6. K. Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” arXiv:1406.1078, 2014.
7. S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
8. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.
9. H. Chen et al., “Machine Learning for Systems and Systems for Machine Learning,” NSDI, 2019.
10. R. Nelson, “Operating System Page Replacement Algorithms,” University Lecture Notes, 2021.
11. J. Kerrisk, “Netlink Sockets: User and Kernel Space Communication,” *The Linux Programming Interface*, No Starch Press, 2010.
12. M. Ferdman et al., “Temporal Prefetching for Memory Locality Optimization,” ISCA, 2011.
13. S. Venkatraman et al., “Using Recurrent Neural Networks to Predict Application Memory Access Patterns,” ACM SIGMETRICS, 2019.
14. S. Garg et al., “RESAT-X: A Reversible Smoothing & Amplitude Transform for PFN Delta Sequences,” (Technical Annex, Future Scope – V4 Enhancement), 2025.

MODEL SUMMARY OUTPUT

This appendix contains the complete architecture summary of the V3 multi-head GRU model used for PFN slice prediction. The model consists of multiple TimeDistributed embedding heads, bidirectional GRUs, auxiliary embeddings, dense projection layers, and six output branches (TOP, S4, S3, S2, S1, S0).

Model Architecture Summary

[Full Keras model summary inserted exactly as generated]

Layer (type)	Output Shape	Param #	Connected to
...			
Total params: 1,416,824 (5.40 MB)			

The model summary demonstrates:

- 128-dimensional time-distributed embeddings,
- Bi-directional GRU layers for deep temporal modeling,
- Auxiliary embeddings for PID, folio index, mapping hash, and latency,
- Multi-head output: TOP (binary), S4–S0 (multi-class),
- KL-consistency head for stabilization of TOP predictions.

KERNEL MODULE SNIPPETS

This appendix contains representative excerpts from the Linux kernel module. These snippets illustrate PFN capture, event packaging, and kprobe attachment logic.

Kprobe Registration

```
/* ----- Module Init/Exit ----- */
static struct kprobe kp_swap_start = {
    .symbol_name = "do_swap_page",
    .pre_handler = do_swap_pre,
};

static struct kprobe kp_swap_done = {
    .symbol_name = "finish_fault",
    .post_handler = finish_fault_post,
};

static struct kprobe kp_swap_start_alt = {
    .symbol_name = "__do_swap_page",
    .pre_handler = do_swap_pre,
};
```

These probes capture:

- Major fault entry,
- Fault completion and PFN extraction,
- Kernel memory mapping behaviour.

Victim Tracking Snippet

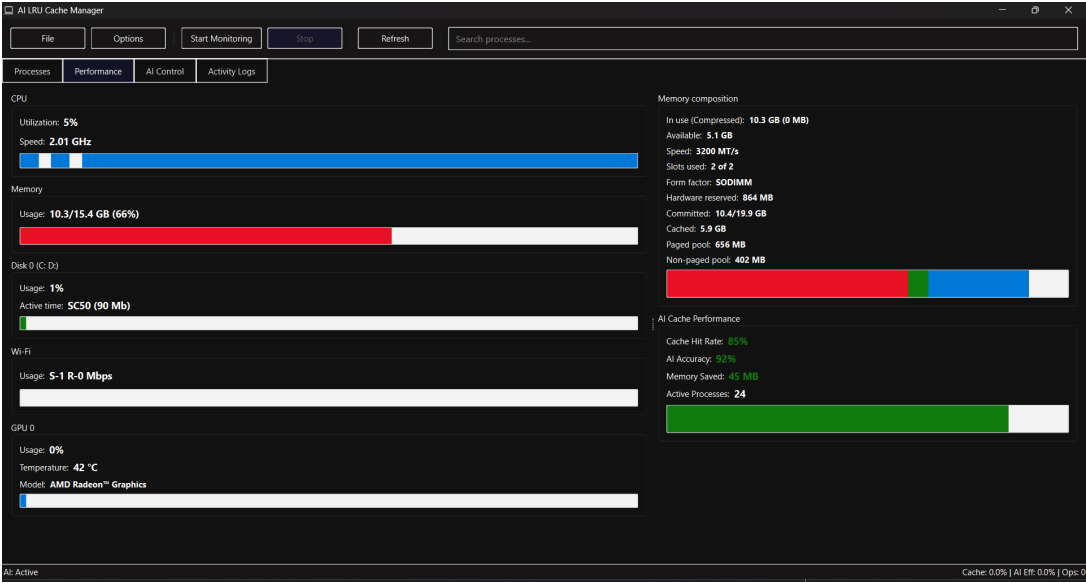
```
// Kprobe for shrink_folio_list (victim selection)
static struct kprobe kp = {
    .symbol_name = "shrink_folio_list",
};
```

Victim detection is anchored inside the `free_it:` label of `shrink_folio_list`, enabling identification of candidate eviction frames.

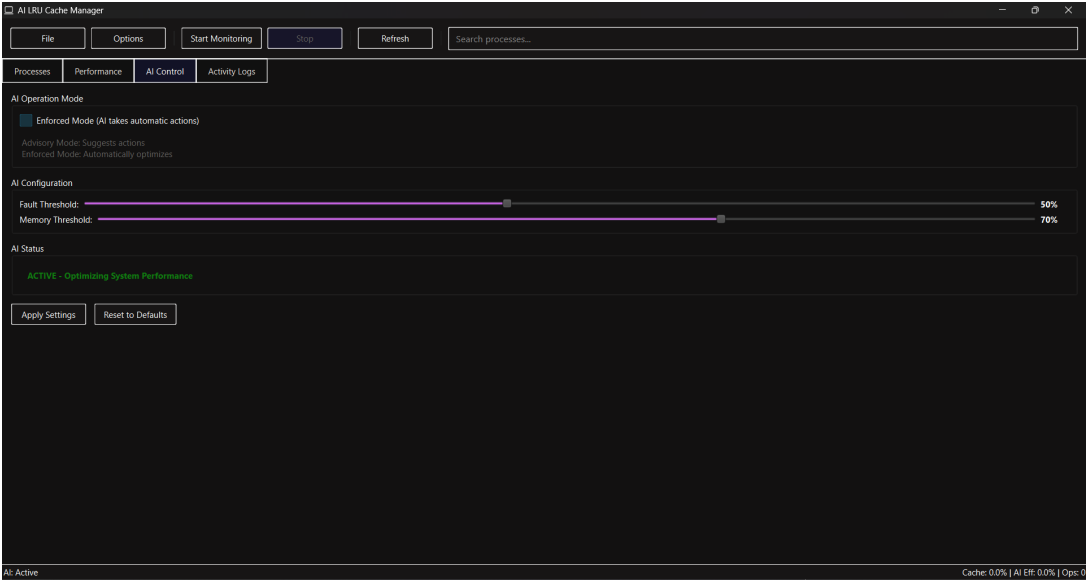
GUI SCREENSHOTS

The following screenshots illustrate the real-time GUI built using Python and PySide6. The application provides live monitoring of kernel events, PFN movement graphs, prediction overlays, and system statistics.

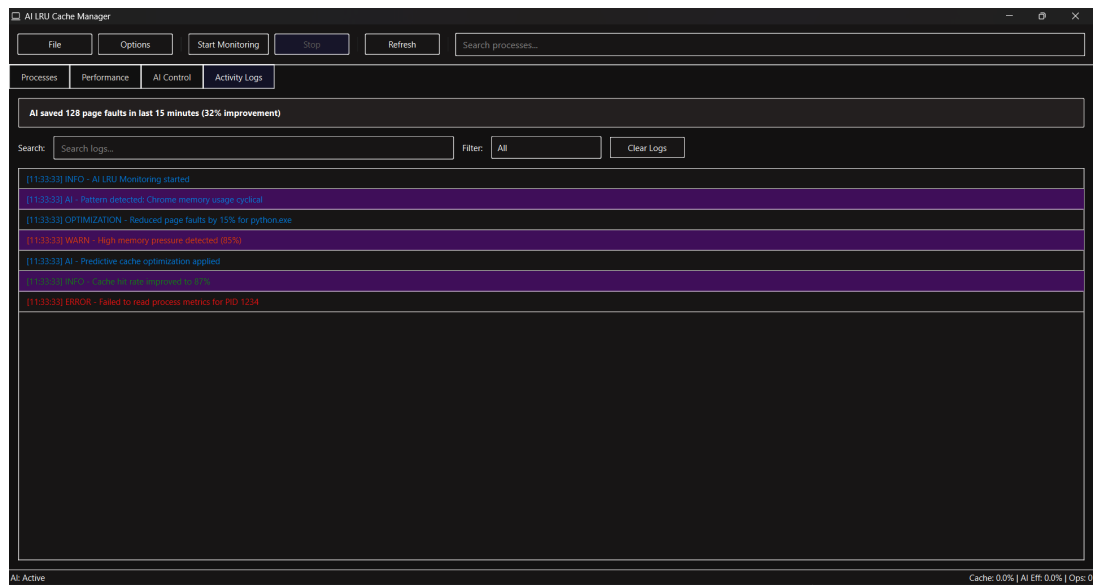
Dashboard View



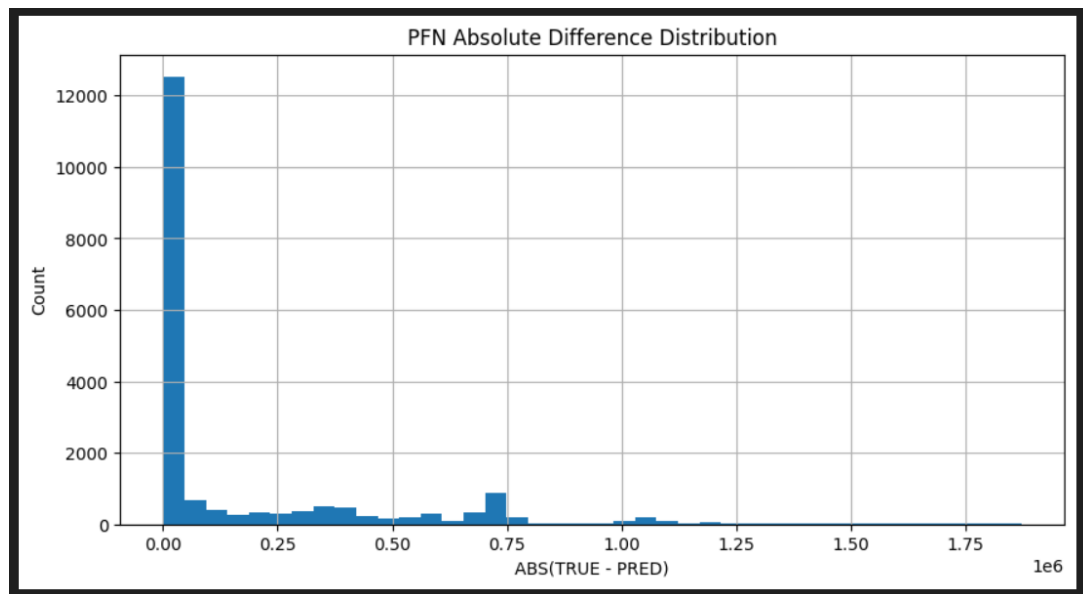
AI Insights Panel



Kernel Activity Tracker



Prediction Overlay Graph

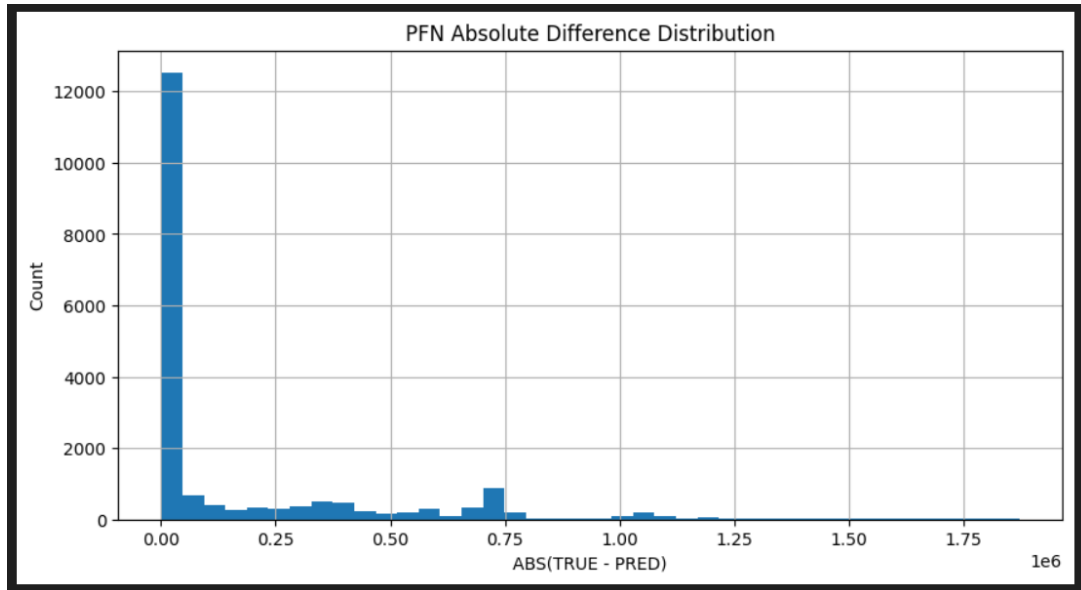


ADDITIONAL GRAPHS

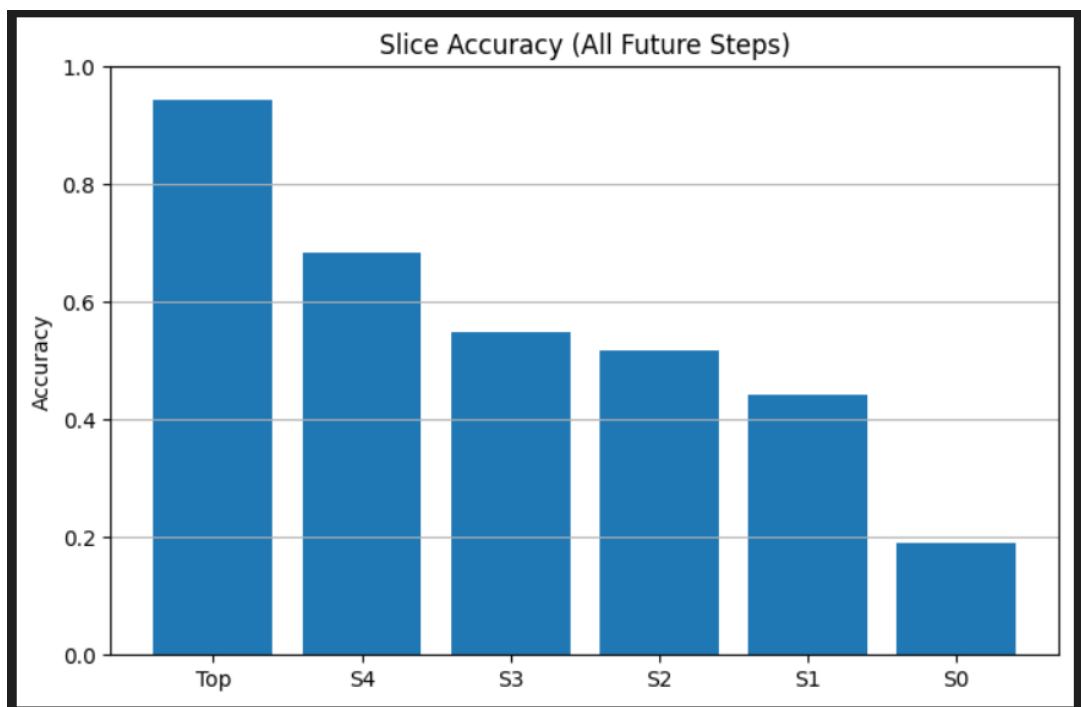
This appendix provides supplementary visualizations collected during testing and analysis.

PFN Curve Sample

The following plot shows a live PFN major-fault curve captured from the kernel, demonstrating the non-linear and spiky nature of PFN access patterns:



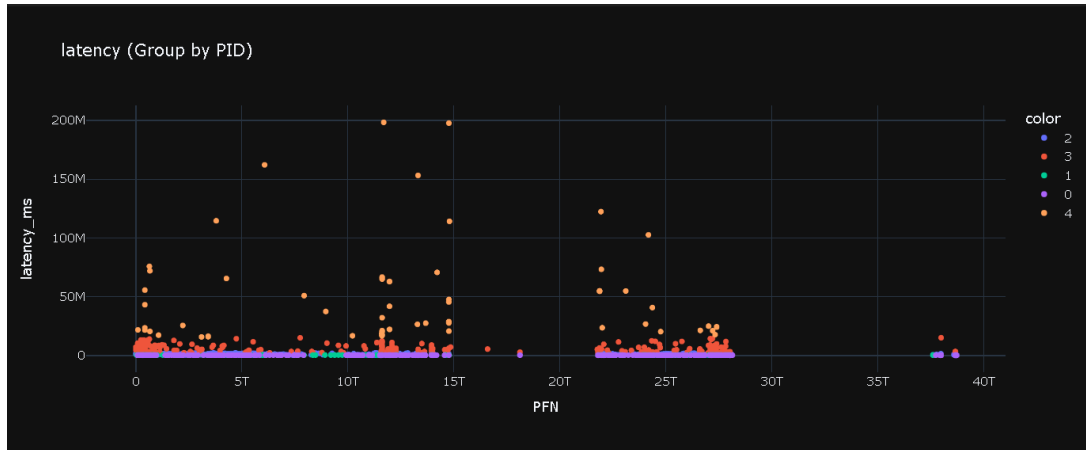
Predicted vs Actual PFN Comparison



This graph highlights:

- Strong alignment in upper slices (TOP, S4),
- Moderate alignment in middle slices (S3, S2),
- Variability in lower slices (S1, S0),
- Region-level correctness even when lower slices fluctuate.

Event Latency Distribution



This plot visualizes the latency (ns) distribution of PFN major faults captured via `finish_fault`.

Workload Comparison Charts

- Browser workloads,
- Multithreaded compiler workloads,
- IO-heavy workloads.

Each workload reveals distinct PFN movement profiles, validating the importance of predictive modeling.